

Problem Solving Strategies and Computational Approaches SCS1304

Handout 3 : Time Complexity- Part I

Prasad Wimalaratne, PhD

Develop Design and Analysis Skills of Algo as a CS student?

Understanding algorithm **design techniques and the analysis of efficiency** helps you develop solutions that are scalable and efficient.

1. Learn Basic Data Structures – Arrays, Linked Lists, Stacks, Queues, Trees, Graphs etc.
2. Study Algorithm Design Paradigms – Divide & Conquer, Greedy, Dynamic Programming etc
3. Practice Complexity Analysis – Solve problems with different Big O notations.
4. Apply in Coding Platforms – LeetCode, HackerRank, Codeforces.
5. Explore Advanced Topics – Graph theory, string algorithms, computational geometry.

Why Analyze an Algorithm?

1. The most straightforward reason for analyzing an algorithm is to discover its characteristics in order to **evaluate its suitability for various applications or compare** it with other algorithms for the same application.
2. Moreover, the analysis of an algorithm can help us **understand** it better, and can **suggest informed improvements**.
3. Algorithms tend to become **shorter, simpler, and more elegant** during the **analysis process**

Importance of Analyses in Algorithm Evaluation

- Understanding these three facets of algorithm performance is essential for a number of reasons:
 1. **Comprehensive Performance Assessment**: Together, these analyses offer a complete picture of an algorithm's efficiency. While the worst case analysis provides a guarantee of sorts about the algorithm's limits, the best and average cases offer a more complex view of its everyday performance.
 2. **Informed Algorithm Selection**: In software development, choosing the right algorithm for a given task is crucial. These analyses help in selecting the most appropriate algorithm based on its expected performance in various scenarios.
 2. **Resource Allocation**: Especially in systems with limited resources, knowing the worst case resource needs is vital. It helps in ensuring that the system can handle the most demanding scenarios.

Importance of Analyses in Algorithm Evaluation

- 4. **Realistic Expectations**: While **worst case** analysis is critical, it can sometimes be **overly pessimistic**. Average case analysis, on the other hand, provides a **more realistic expectation of how the algorithm will perform in everyday use**.
- 5. **Algorithm Optimization**: Understanding the different performance scenarios can guide developers in optimizing algorithms. For instance, if the worst case performance is significantly lagging, efforts can be directed towards mitigating these cases.
- Best, worst, and average case analyses are important tools in algorithm design and evaluation. They provide vital insights into the behavior of algorithms under various conditions, **guiding developers in creating more efficient and effective solutions**.

Analysis of Algorithms

■ Time complexity analysis

- Determination of how many times the **basic operation** is done for each value of the **input size**.
- Should be independent of CPU, OS, Programming languages...

■ Metrics

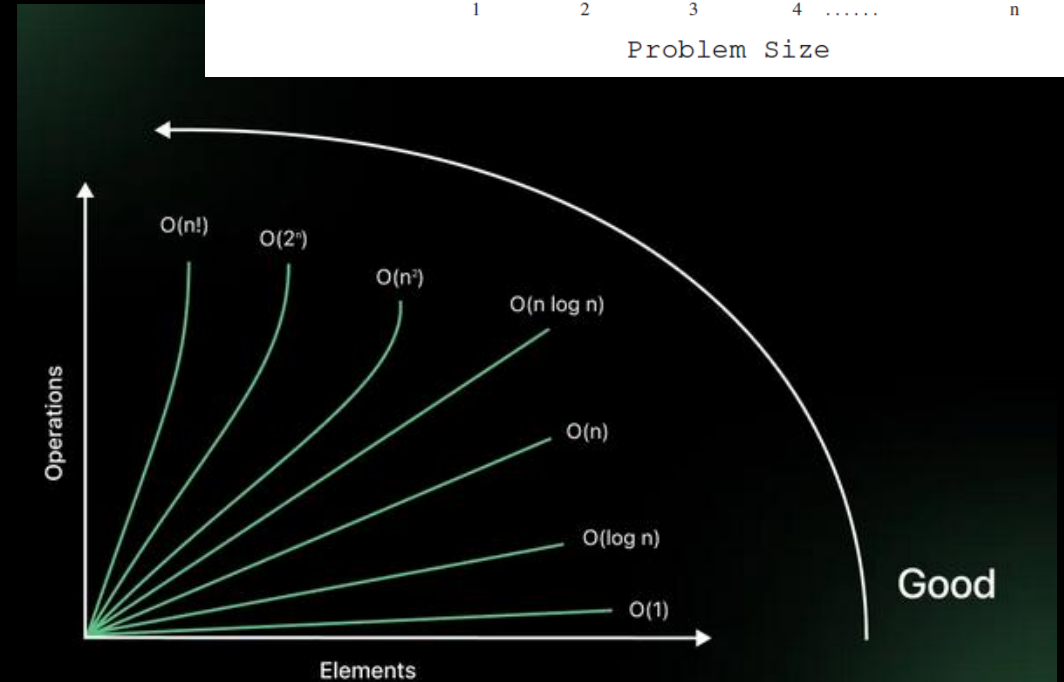
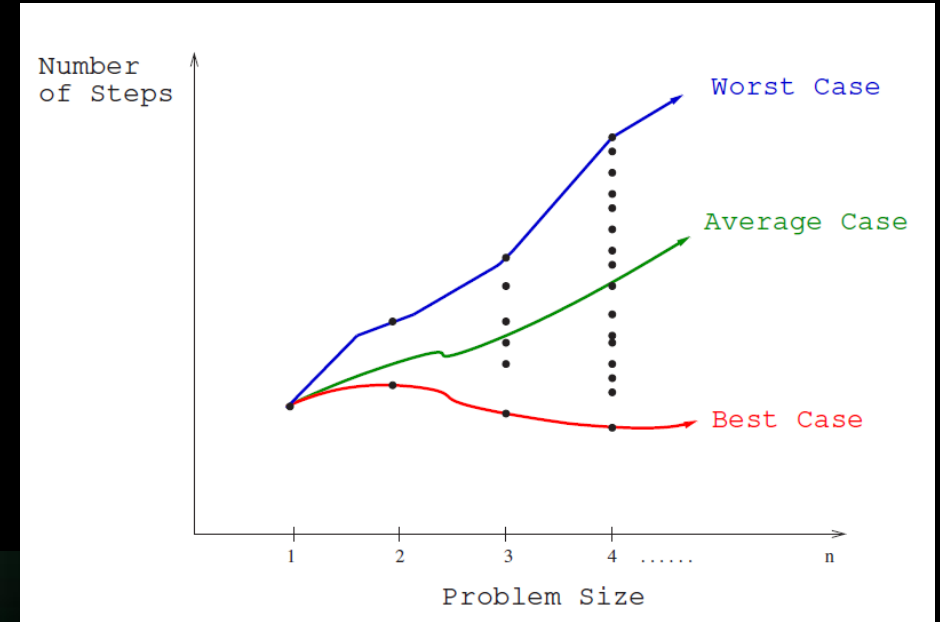
1. Basic operation
 - Comparisons, assignments, etc.
2. Input size /elements
/problem size

e.g

- The number of elements in an array
- The length of a list
- The number of columns and rows in a matrix
- The number of vertices and edges in a graph

Analysis of Algorithms ctd..

1. Worst-case analysis
2. Average-case analysis
3. Best-case analysis



Worst Case Analysis (Pessimistic Scenario) - Mostly used

- This is the exact **opposite of the best case**.
- It considers **the most challenging OR demanding situation for the algorithm**.
- The worst case analysis provides **an upper bound on the time OR resources** needed for the algorithm's completion which is good information.
- Example :
 - A worst-case scenario for a **sorting algorithm would be sorting a list that is in completely reverse order**.
 - For **Linear Search**, the worst case happens when the element to be searched (x) is the **last element or not present in the array**.
 - When x is not present, the search() function compares it with all the elements of array one by one. Therefore, the worst-case time complexity of the linear search would be $O(n)$.
- We must know the case **that causes a maximum number of operations to be executed**.

Significance of Worst Case Scenarios

- Worst case analysis is **a fundamental aspect of evaluating algorithm performance.**
- It involves determining how an algorithm **behaves under the most adverse conditions.**
- This type of analysis examines the **maximum number of operations** an algorithm might perform or **the maximum amount of resources** it might require.
- It answers a critical question: "**In the least favorable scenario, how badly could this algorithm perform?**"
- This is crucial for applications where **predictability and reliability** are very important, such as in mission critical applications/life critical medical applications or important financial applications.

Ex. List three

- a) mission critical applications
- b) life critical medical applications
- c) Important financial applications

Worst Case Examples from Widely-Used Algorithms

1. **Linear Search**: In a linear search algorithm, the worst case occurs when the element being searched for is **not present in the array** or is **at the very end**. The algorithm has to check each element, leading to a time complexity of $O(n)$.
2. **Sorting Algorithms** (like Quick Sort): For Quick Sort, the worst case happens when the pivot element selected at each step is the smallest or largest element, leading to **imbalanced partitions**. This results in a time complexity of $O(n^2)$. (Detailed under sorting example later lecture)
3. **Graph Algorithms** : In **graph search algorithm for finding the shortest path**, the worst case is when the algorithm has to **traverse all vertices and edges in the graph**, leading to a time complexity of $O(V^2)$ for basic implementations, where V is the number of vertices. (you will learn algorithms like Dijkstra's Algorithm in later lessons)

Importance of Worst Case Analysis in Algorithm Design

- Worst case analysis is often considered the **most crucial** aspect of algorithm design for several reasons:
 - 1. **Reliability and Robustness**: It ensures that an algorithm's performance is predictable **even in the most challenging scenarios**, which is critical for maintaining system stability and reliability.
 - 2. **Resource Allocation**: Understanding the worst case helps in adequately provisioning resources such as **memory and processing power**, ensuring that the system can handle the most resource-intensive scenarios.

Importance of Worst Case Analysis in Algorithm Design

- 3. **Risk Management**: In critical applications, worst case scenarios help identify potential **risks and bottlenecks**, allowing developers to mitigate these risks early in the design phase.
- 4. **Performance Guarantees**: Worst case analysis offers a conservative estimate of algorithm performance, **providing a safety net for both developers and users**. It is particularly useful **when the input data characteristics are unknown or highly variable**.
- 5. **Comparison and Selection of Algorithms**: In many cases, especially in **competitive programming or system optimization**, the choice of an algorithm is based on its worst case performance, as it provides a **clear metric for comparison**.

Importance of Worst Case Analysis in Algorithm Design

- Worst case analysis is a **vital tool** in algorithm evaluation, providing essential insights into the limits of an algorithm's performance.
- By considering the most challenging scenarios, developers can design algorithms that are not **only efficient under typical conditions but** also robust and **reliable under extreme conditions**.
- This comprehensive understanding is **key to creating high-performance, resilient systems and applications**.

Average Case Analysis (Realistic Scenario)

- Falling **between the best and worst extremes**, the average case analysis attempts to ascertain the algorithm's performance under '**typical**' or '**average**' conditions.
- It is **often more complex to compute** as it requires understanding of **how the inputs are distributed**.
 - E.g The average case for the **linear search** would involve calculating the **expected number of comparisons for a randomly chosen target** in the array.
- The average case analysis is **not easy to do in most practical cases and it is rarely done**. In the average case analysis, we must know (or predict) the mathematical distribution of all possible inputs.

Average Case Analysis (Realistic Scenario)

- In average case analysis, we **take all possible inputs and calculate the computing time for all of the inputs.**
- **Sum** all the calculated **values and divide the sum by the total number of inputs.**
- We must know (or predict) the distribution of cases.
- For the **linear search problem**, let us assume that all cases are **uniformly distributed** (including the case of x not being present in the array). So we sum all the cases and divide the sum by **$(n+1)$** .

Limitations in Applicability of Average Case

- The **usefulness** of average case analysis can **be limited** if the **actual input distribution significantly differs from the assumed distribution**, or if the distribution of inputs is unknown.
- Average case analysis plays a crucial role in understanding and predicting the practical performance of algorithm .
- **While it may be complex to derive, its insights are invaluable** **in** guiding the development of efficient and realistic algorithms suited for the typical conditions they will encounter in actual use.

Best Case Analysis (Optimistic Scenario) - Very Rarely used

- This refers to the scenario where the algorithm performs its task under the **most favorable conditions**.
- It represents the **minimum time OR resources required** for the algorithm to complete its execution.
 - For example, in the best case, a search algorithm might find the target element in the **first place** it looks.
- In the best-case analysis, **we calculate the lower bound on the running time of an algorithm**. We must know the case that causes a minimum number of operations to be executed.
- In the linear search problem, **the best case occurs when x is present at the first location**. The number of operations in the best case **is constant (not dependent on n)**. So time complexity in the best case would be 1
- Guaranteeing a lower bound on an algorithm **does not provide any information as in the worst case, an algorithm may take years to run**.

Examples of Best Case Analysis in Common Algorithms

- 1. Linear Search: The best case in a linear search algorithm occurs when the target element is the **first element in the list**. Here, the algorithm **only makes one comparison**, regardless of the **list's size**.
- 2. Binary Search: For binary search, the best case happens when the **middle element (first checked) is the target element**. Even for large arrays, the search completes after just one comparison.
- 3. Sorting Algorithms (like Bubble Sort): In bubble sort, the best case occurs when the array is **already sorted**. The algorithm goes through the list once without making any swaps, thus completing in linear time.

Why Best Case Analysis Can Be Misleading in Practical Applications

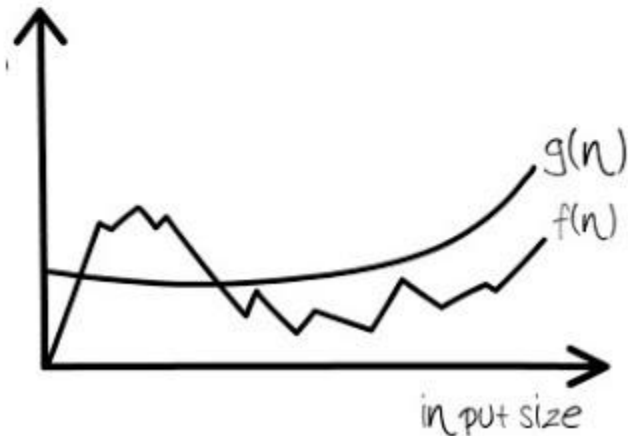
- While best case analysis provides an optimistic view of an algorithm's potential efficiency, it can be misleading for several reasons:
 1. **Unrealistic Expectations:** The best case scenario often represents a highly idealized and rare situation. Relying solely on this analysis might create unrealistic expectations about the algorithm's typical performance.
 2. **Not Reflective of Average Performance:** In most real-world applications, inputs to algorithms are not systematically arranged to meet the best case criteria. Thus, the best case does not accurately reflect the average or expected performance of the algorithm.
 3. **Overemphasis on Optimistic Outcomes:** Focusing too much on best case scenarios can lead to neglecting more common and challenging cases, resulting in a lack of preparation for the algorithm's actual operational environment.
 4. **Resource Allocation and Planning:** In system design and resource allocation, planning based on best case scenarios can result in inadequate resource provisioning, potentially leading to performance issues in average or worst-case scenarios.

Why Best Case Analysis Can Be Misleading in Practical Applications

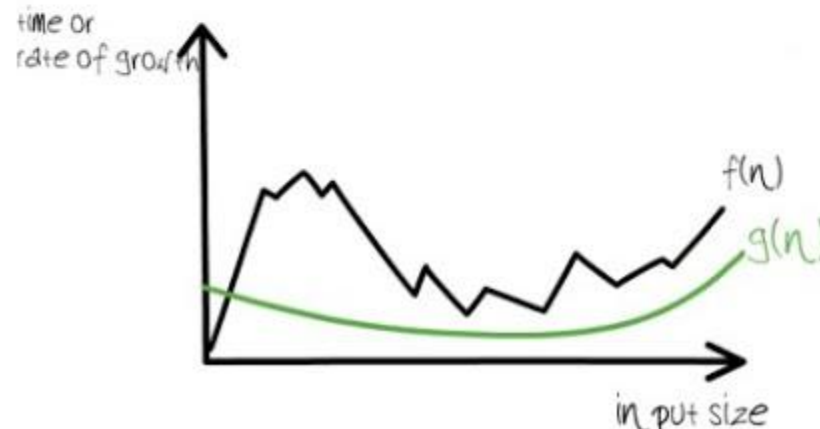
- While best case analysis is a part of a comprehensive algorithm evaluation, it should be **considered alongside worst and average case analyses to gain a realistic understanding** of an algorithm's performance.
- It serves more as a **theoretical benchmark rather than a sole indicator** of practical efficiency, and its limitations must be recognized in the broader context of algorithm design and application.

TIME COMPLEXITY AND ASYMPTOTIC NOTATION

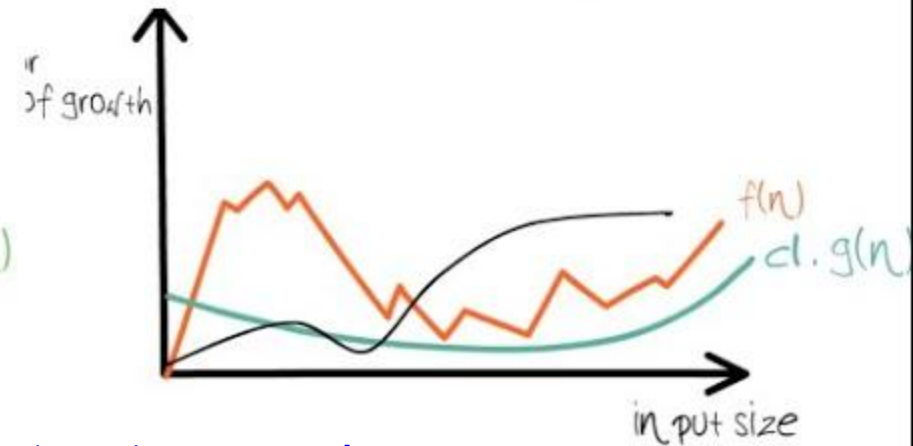
WORST CASE



BEST CASE



AVERAGE CASE



Comparative Analysis of Best, Worst, and Average Cases

- Understanding the differences and relative importance of best, worst, and average case analyses is crucial for a comprehensive evaluation of algorithms.

1. Best Case Analysis:

- **Focus:** **Optimistic** scenario where the algorithm performs with the least effort.
- **Usage:** Often used for theoretical benchmarks and understanding the **lower limit of an algorithm's efficiency**.
- **Limitation:** Can be misleading as it does not reflect typical or challenging scenarios.

2. Worst Case Analysis:

- Focus: **Pessimistic** scenario considering the most demanding input.
- Usage: Crucial for ensuring **reliability and performance guarantees**, especially in critical applications.
- Advantage: Provides an upper bound on resource requirements, ensuring that the algorithm can handle the toughest scenarios.

3. Average Case Analysis:

- Focus: **Realistic** scenario, taking into account the probability distribution of all possible inputs.
- Usage: Offers a balanced view of algorithm performance in typical use.
- Complexity: Requires knowledge of or assumptions about input distribution, making it more complex to calculate.

Time Complexity Analysis: (In Big-O notation)

- **example : Linear Search**
 - **Best Case:** This will take place if the element to be searched is on the first index of the given list. So, the number of comparisons, in this case, is 1.
 - **Average Case:** This will take place if the element to be searched is on the middle index of the given list.
 - **Worst Case:** This will take place if:
 - The element to be searched is on the last index
 - The element to be searched is not present on the list

O

Big Oh

Upper Bound

Worst Case

Ω

Omega

Lower Bound

Best Case

Θ

Theta

Upper & Lower
Bound

Average 'ish'

Popular Notations in Complexity Analysis of Algorithms

- 1. Big-O Notation
- We **define an algorithm's worst-case time complexity** by using the Big-O notation, which determines the set of functions grows slower than or at the same rate as the expression. Furthermore, it **explains the maximum amount of time an algorithm requires to consider all input values**.
- 2. Omega Notation
- **It defines the best case of an algorithm's time complexity**, the Omega notation defines whether the **set of functions will grow faster or at the same rate as the expression**. Furthermore, it explains the minimum amount of time an algorithm requires to consider all input values.
- 3. Theta Notation
- **It defines the average case of an algorithm's time complexity**, the Theta notation defines when the set of functions lies in both $O(\text{expression})$ and $\Omega(\text{expression})$, then Theta notation is used. This is how we define a time complexity average case for an algorithm.

Problem Solving Strategies and Computational Approaches SCS1304

Handout 4 : Time Complexity & Asymptotic notation - Part II

Prasad Wimalaratne PhD

Time complexity

- Time complexity is a programming term that quantifies the amount of time it takes a sequence of code or an algorithm to process or execute **in proportion to the size of input**.
- It will not look at an algorithm's overall execution time.
- Rather, it will provide data on the **variation (increase or reduction) in execution** time when the number of operations in an algorithm increases or decreases.
- Asymptotic notation is a mathematical tool used to describe the efficiency of an algorithm as the input size approaches infinity.
- It provides a way to express the **growth rate of an algorithm's runtime or space requirement**.

Exact Runtime Analysis Vs. Asymptotic Analysis

Feature	Exact Runtime Analysis	Asymptotic Analysis
Definition	Determines the precise time taken by an algorithm for a given input size.	Describes the upper, lower, or tight bounds on growth rate.
Focus	Machine-dependent execution time.	Growth rate as $n \rightarrow \infty$.
Example	"This sorting algorithm takes 50ms for $n=1000$."	"This sorting algorithm runs in $O(n \log n)$ time."
Considerations	Includes constant factors, CPU speed, and system performance.	Ignores constants and lower-order terms.
Use Case	Used when actual performance measurement is needed.	Used to compare algorithms theoretically.

Asymptotic Notation

- Asymptotic notation is a way to describe the performance of algorithms, specifically how **their runtime or memory usage (complexity)** grows as the input size increases.
- It focuses on the dominant behavior of the algorithm as the input approaches infinity, ignoring constant factors and lower-order terms.
- Essentially, it provides a shorthand for understanding the efficiency of an algorithm in the "long run"
- **Big-O notation** represents the **upper bound of the running time** of an algorithm.
- Thus, it gives the **worst-case complexity** of an algorithm.

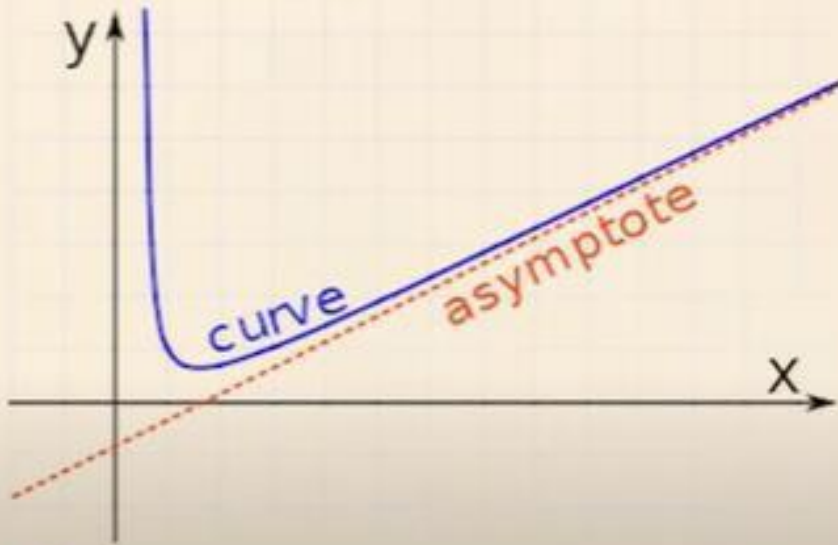
Asymptotic Notations

What does asymptotic mean?

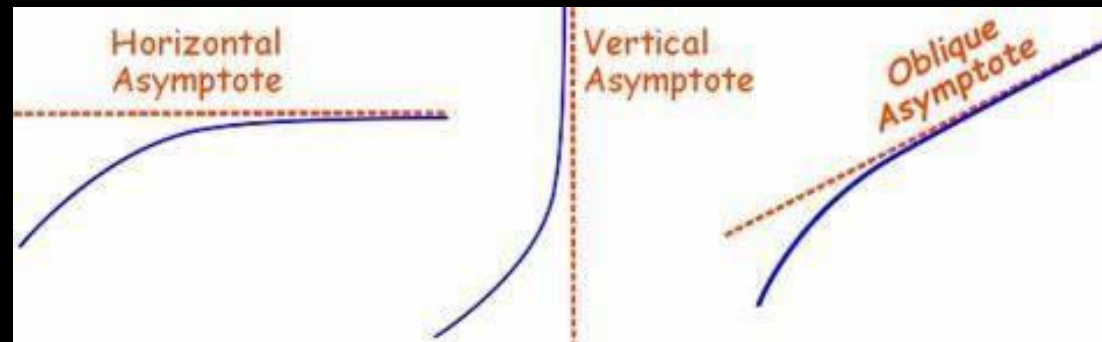
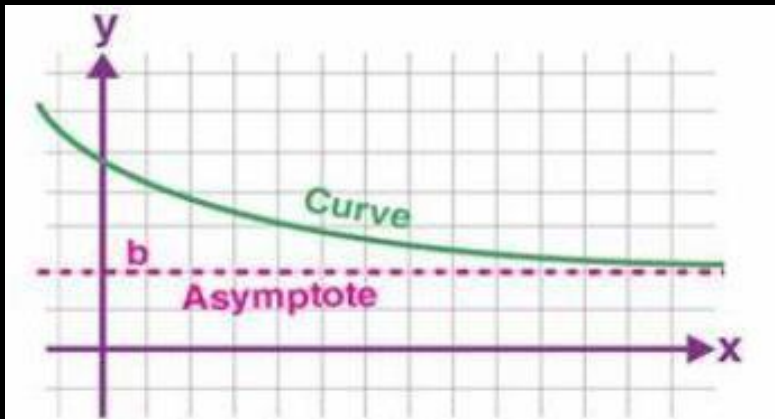
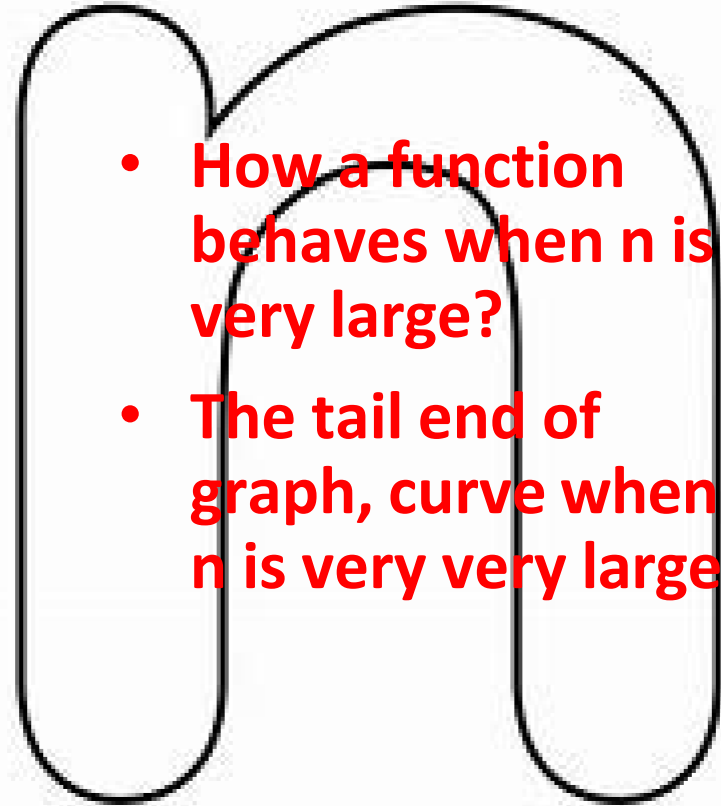
- The word **asymptotic** means **approaching a certain value** which could be considered as the limit.
 - The value **can range from any finite natural number to an infinite value**.
- Definition

“Asymptotic Notation is used to decide the asymptotic running time of an algorithm, **when the input size n is very large, i.e, $n \rightarrow \infty$** ; ‘ n ’ belongs to the set of natural numbers. “
- An asymptote of a curve is a line that the curve approaches but never actually touches or intersects.
 - It is a line that the **curve gets infinitely close** to as it extends towards infinity

Asymptotic



- How a function behaves when n is very large?
- The tail end of graph, curve when n is very very large



Rigorous Definition to **Order**: Big O

- Definition: (Asymptotic Upper Bound)
 - For a **given** complexity function $f(n)$, $O(f(n))$ is **the set of complexity functions $g(n)$** for which **there exists some positive real constant C** and **some non-negative integer N** such that for all $n \geq N$,

$$g(n) \leq c \times f(n)$$

- $g(n) \in O(f(n))$

****Note the word SOME in definition. i.e not unique**

Refer: Asymptotic Notation (Play List)

<https://www.youtube.com/playlist?list=PLQfaHkBRINswUNbAHUOwi1tTxFxi1xUII>

Big O

Big O Notation (O) describes the **upper bound or worst-case growth rate** of an algorithm's time or space complexity **as the input size (n) increases**.

Definition

An **algorithm is $O(f(n))$** if its **running time does NOT grow faster than a constant multiple of $f(n)$** for large values of n .

Asymptotic Notation

Motivation

$3n^2 + 4n - n + 1$ times

```
1: for i = n to  $3n^2 + 4n$  do
2:   for j = 1 to  $7n + 3$  do
3:      $x = x + i - j$ 
4:   end for
5: end for
```

c } $7n + 3$ times

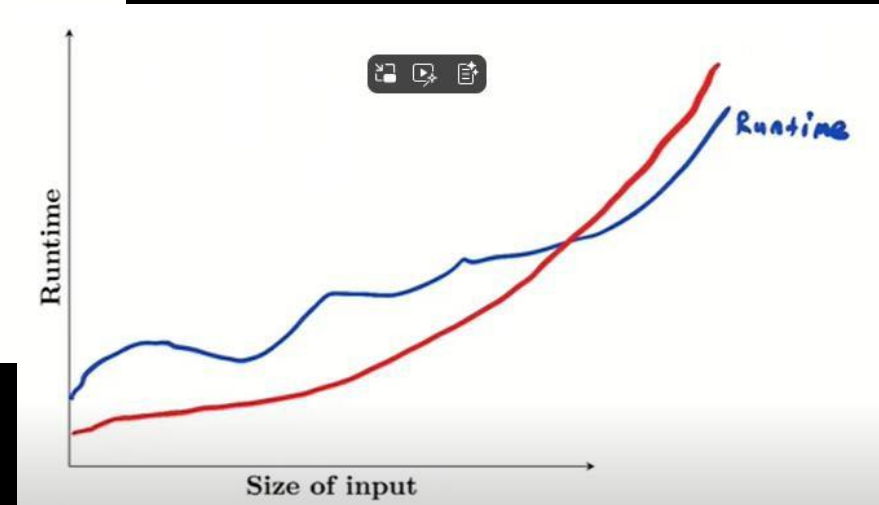
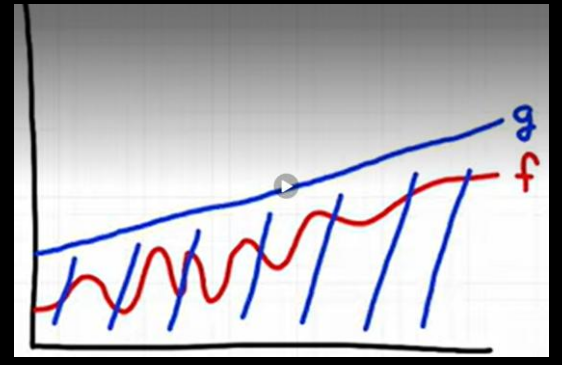
$$\text{Runtime: } (3n^2 + 3n + 1)(7n + 3)c$$

**Looping with inclusive range from start to end, $\Rightarrow$ iterations = end - start + 1

Fastest growing term

$$\text{Runtime: } (3n^2 + 3n + 1)(7n + 3)c \in \Theta(n^3)$$

$\approx cn^3$



Refer: Asymptotic Notation (Play List)

<https://www.youtube.com/playlist?list=PLQfaHkBRINswUNbAHUOWi1tTxFxi1xUII>

Meaning?

- Big O - what the **slowest and most space** an algo can use
 - Omega – what the **fastest and least space** an algo can use
 - *** $g(n)$ is **about the behavior** than a mathematical function.
-
- Big O notation is usually expressed using the tightest (closest) asymptotic upper bound.
 - Although many larger upper bounds may satisfy the definition, the nearest class of functions provides the most useful description of the algorithm's growth rate.

Upper Bound $O(n)$, Lower Bound and Average Bound of a class of functions

- Big O is written in the closest upper bound not higher though the equality holds
- Nearest class of functions is useful.
- closest (tightest) upper bound is that it gives the most useful description of growth.

For example:

$$T(n) = 3n^2 + 5n + 2$$

It is true that:

$$T(n) = O(n^2)$$

$$T(n) = O(n^3)$$

$$T(n) = O(n^4)$$

Because all of these are valid upper bounds.

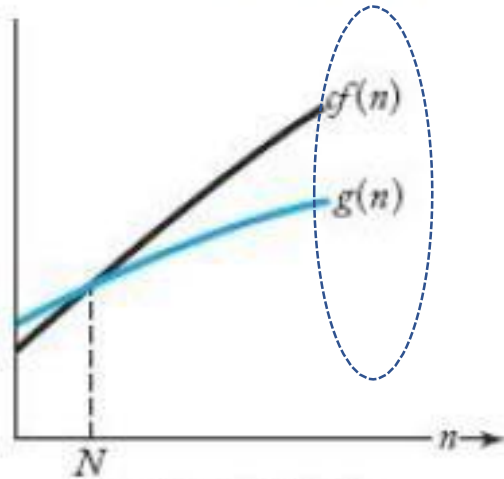
However, $O(n^2)$ is preferred because it is the smallest/common nearest upper-bound class that describes the growth rate accurately.

Asymptotic Notations

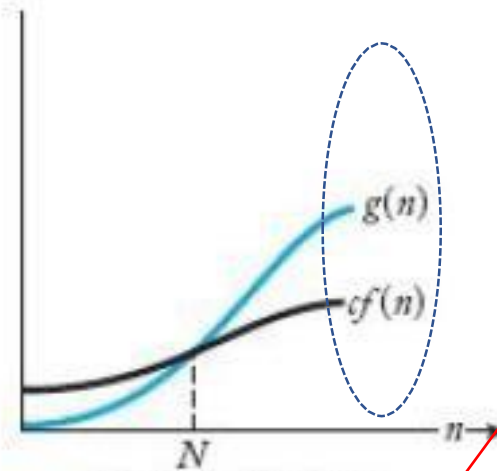
- The asymptotic notation is a defined pattern to measure the performance and memory usage of an algorithm
 - i. Omega Notation “ Ω ”: **best-case** scenario where the time complexity will be **as optimal as possible** based on the input.
 - ii. Theta Notation “ Θ ”: **average-case** scenario where the time complexity will be **the average** considering the input.
 - iii. Big-O Notation “ O ”: **worst-case** scenario is the most important operator to learn because we can measure the worst-case scenario time complexity of an algorithm.

Illustrating “big O,” Ω , and Θ .

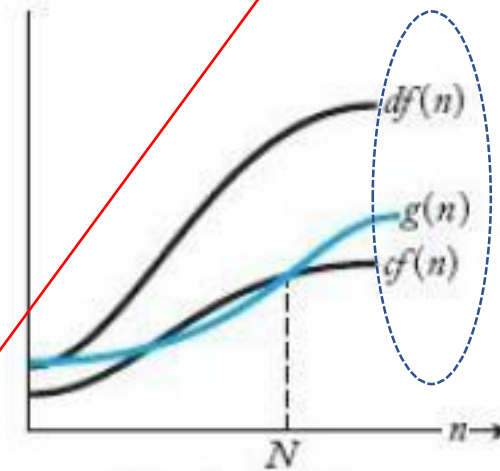
Figure 1.4 Illustrating “big O,” Ω , and Θ .



(a) $g(n) \in O(f(n))$



(b) $g(n) \in \Omega(f(n))$



(c) $g(n) \in \Theta(f(n))$

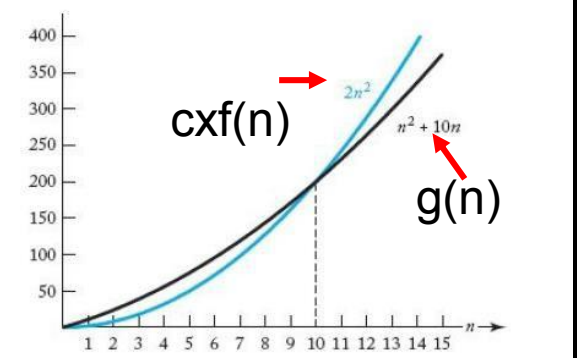
$$n^2 + 10n \leq 2n^2.$$

$$g(n) \leq c \times f(n)$$

We can therefore take $c = 2$ and $N = 10$ in the definition of “big O” to conclude that

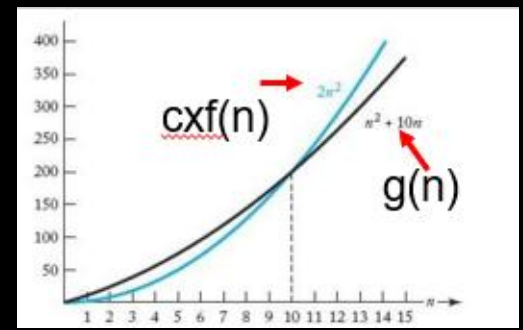
$$n^2 + 10n \in O(n^2).$$

$$g(n) \text{ is in } O(n^2)$$



Meaning? This means that if $g(n)$ is the time complexity for some algorithm, eventually the running time of the algorithm will be at least as fast as quadratic.

Illustrating “big O,” Ω , and Θ .



- If, for example, $g(n)$ is in $O(n^2)$, then eventually $g(n)$ lies beneath some pure quadratic function cn^2 on a graph.
 - This means that if $g(n)$ is the time complexity for some algorithm, eventually the running time of the algorithm will be at least as fast as quadratic.
 - For the purposes of analysis, we can say that eventually $g(n)$ is at least as good as a pure quadratic function.
- “Big O” (and Big Ω , and Θ) are said to describe the asymptotic behavior of a function because they are concerned only with eventual behavior.
- **We say that “big O” puts an asymptotic upper bound on a function.

Example 1.7

We show that $5n^2 \in O(n^2)$. Because, for $n \geq 0$,

$$5n^2 \leq 5n^2,$$

we can take $c = 5$ and $N = 0$ to obtain our desired result.

$$g(n) \leq c \times f(n)$$

Example 1.8

Recall that the time complexity of Algorithm 1.3 (Exchange Sort) is given by

$$T(n) = \frac{n(n-1)}{2}.$$

Because, for $n \geq 0$,

$$\frac{n(n-1)}{2} \leq \frac{n(n)}{2} = \frac{1}{2}n^2,$$

we can take $c = 1/2$ and $N = 0$ to conclude that $T(n) \in O(n^2)$.

$$g(n) \leq c \times f(n)$$

A difficulty students often have with “big O” is that they erroneously think there is some unique c and unique N that must be found to show that one function is “big O” of another. This is not the case at all. Recall that Figure 1.5 illustrates that $n^2 + 10n \in O(n^2)$ using $c = 2$ and $N = 10$. Alternatively, we could show it as follows.

Figure 1.5 The function $n^2 + 10n$ eventually stays beneath the function $2n^2$.

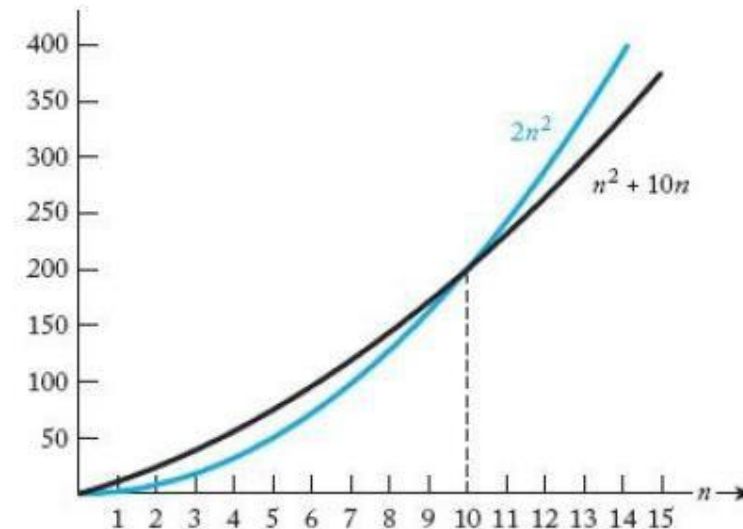
Example 1.9

We show that $n^2 + 10n \in O(n^2)$. Because, for $n \geq 1$,

$$n^2 + 10n \leq n^2 + 10n^2 = 11n^2,$$

we can take $c = 11$ and $N = 1$ to obtain our result.

In general, one can show “big O” using **whatever manipulations seem most straightforward**



In general, one can show “big O” using whatever manipulations seem most straightforward.

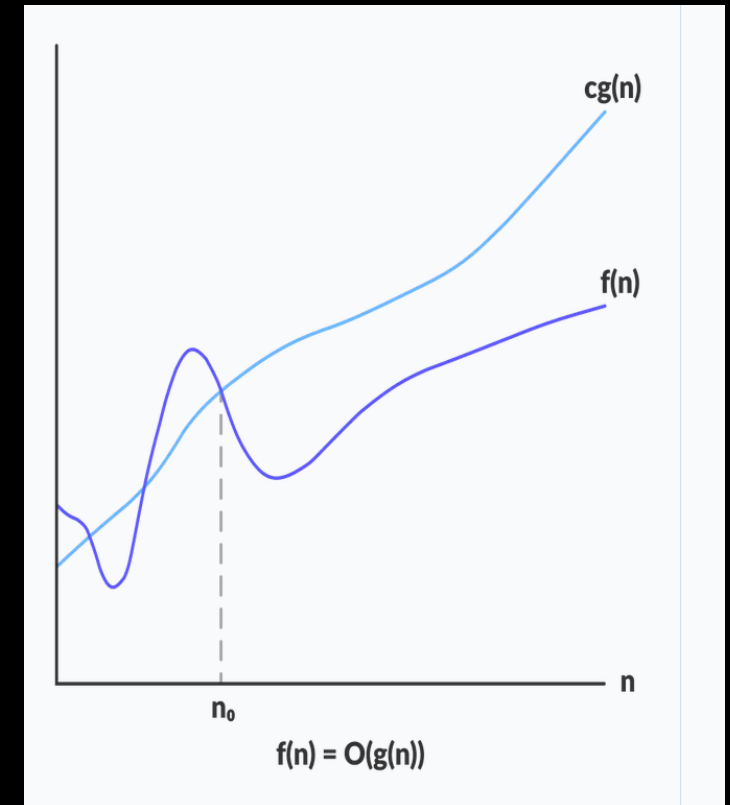
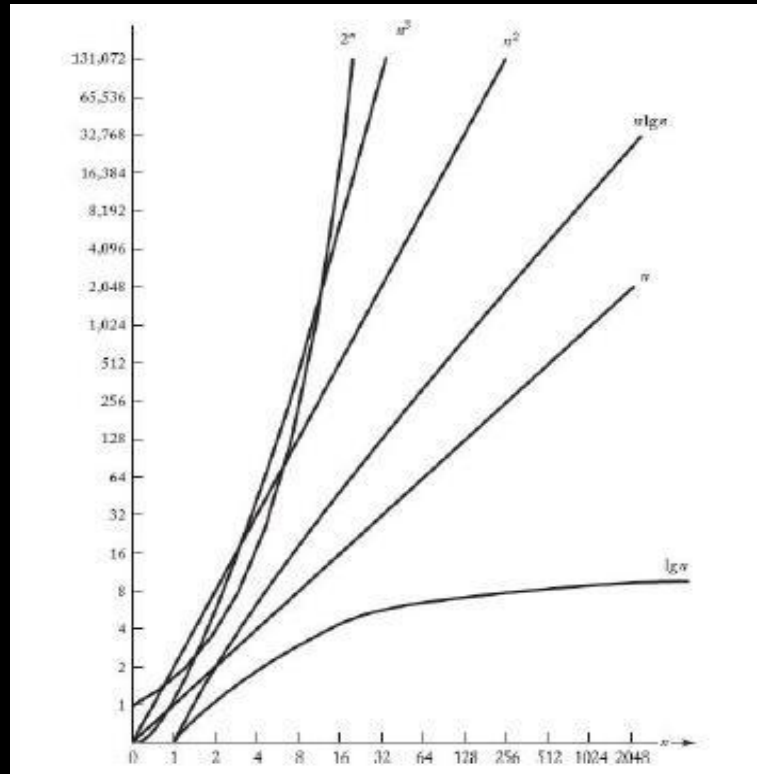
Example 1.9

We show that $n^2 + 10n \in O(n^2)$. Because, for $n \geq 1$,

$$n^2 + 10n \leq n^2 + 10n^2 = 11n^2,$$

we can take $c = 11$ and $N = 1$ to obtain our result.

- The purpose of this last example is to show that the function inside “big O” does not have to be one of the simple functions plotted in Figure 1.3.(below) It can be any complexity function.
- **Ordinarily**, however, we take it to be a simple function like those plotted in Figure 1.3 (below).

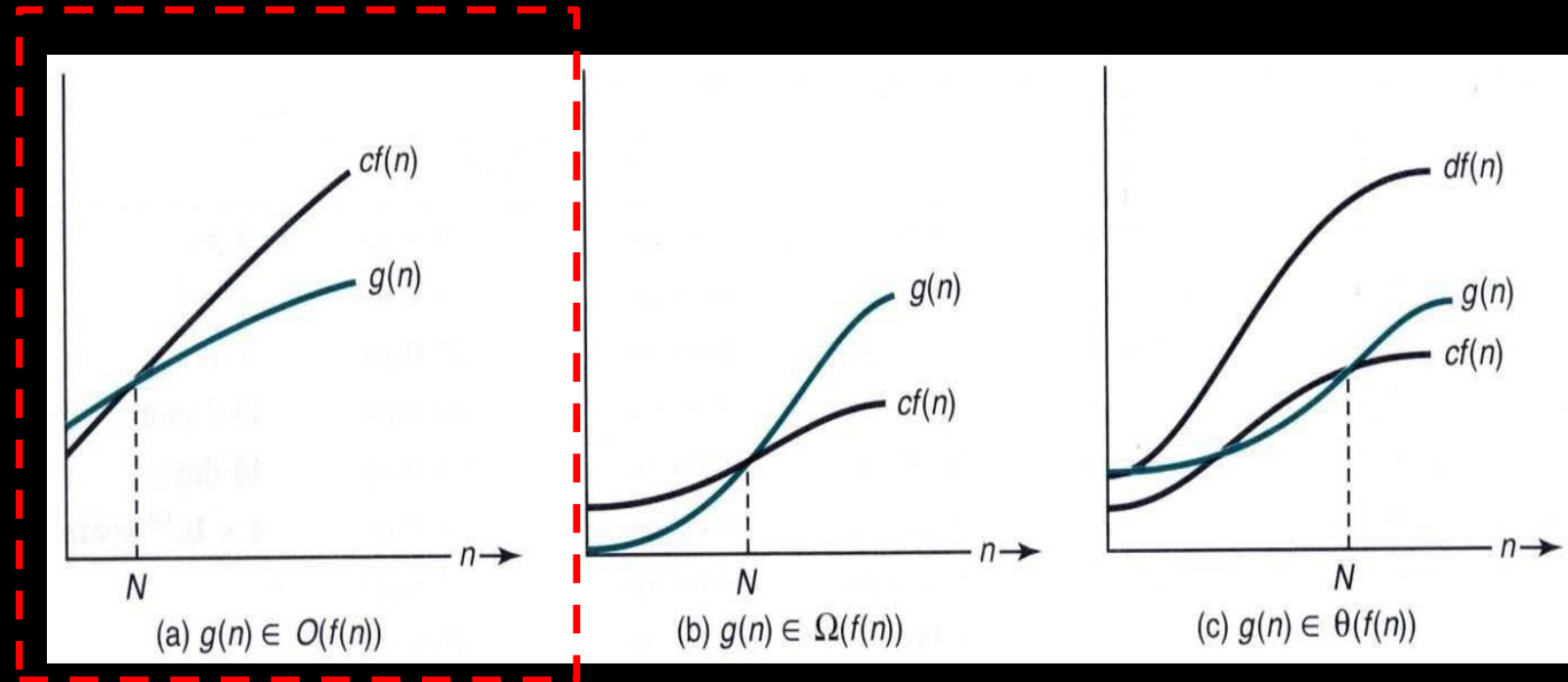


- Refer https://web.mit.edu/16.070/www/lecture/big_o.pdf

Big O Notation: Definition

Meaning of $g(n) \in O(f(n))$

- Although $g(n)$ starts out above $cf(n)$ in the figure, **eventually** it falls **beneath** $cf(n)$ and **stays there**.
- If $g(n)$ is the time complexity for an algorithm, **eventually** the running time of the algorithm will be at least **as good as** $f(n)$
- $f(n)$ is called as an asymptotic upper bound (of *what?*) (i.e. $g(n)$ **can not** run **slower** than $f(n)$, **eventually**)



Big O Notation: Example

- Meaning of $n^2+10n \in O(n^2)$

- Take $c = 11$ and $N = 1$.
- Take $c = 2$ and $N = 10$.
- If n^2+10n is the time complexity for some algorithm, **eventually** the running time of the algorithm will be **at least as fast (good) as n^2**
- *** **$11n^2$ is an asymptotic upper bound** for the time complexity function of **n^2+10n** .

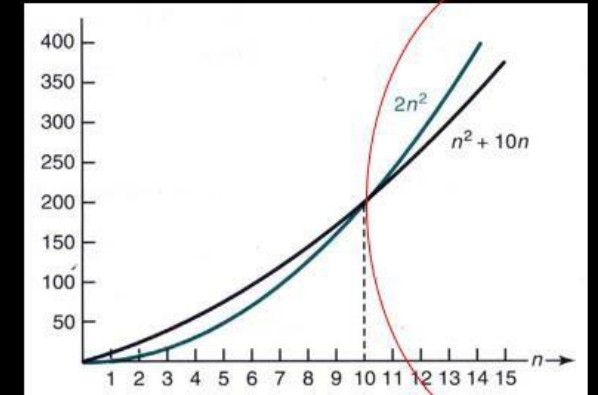
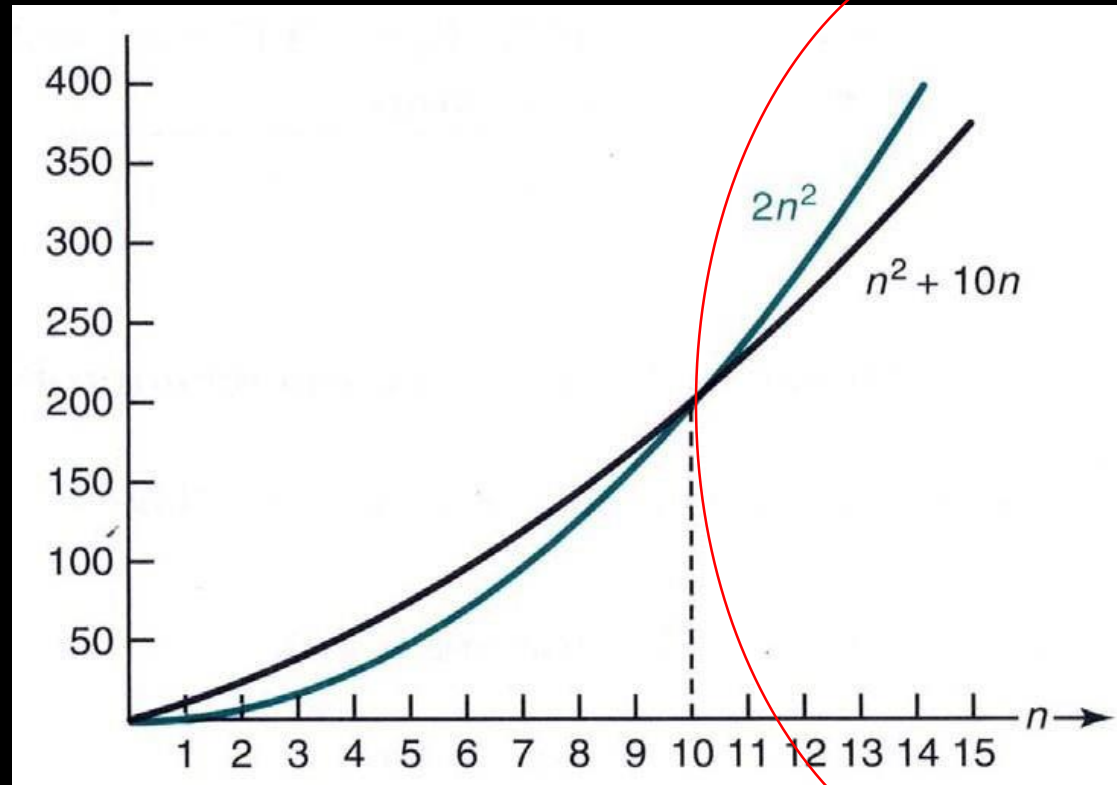
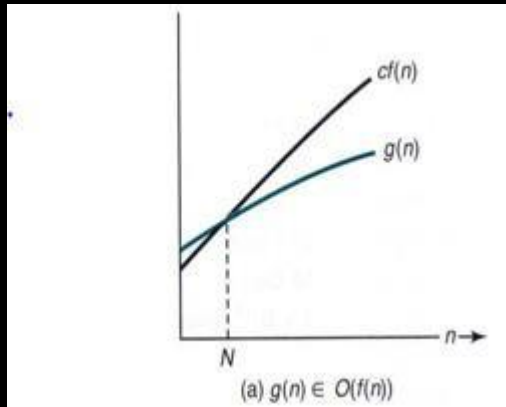
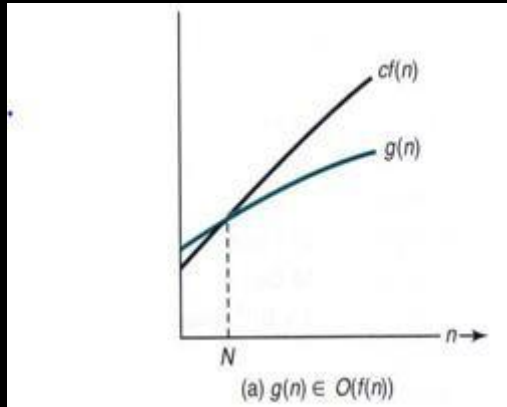


Figure 1.5 The function $n^2 + 10n$ eventually stays beneath the function $2n^2$.



Big O Notation: More Examples



- $5n^2 \in O(n^2)$

- Take $c = 5$ and $N = 0$, then for all n such that $n \geq N$, $5n^2 \leq cn^2$.

- $T(n) = \frac{n(n-1)}{2}$

- Because, for $n \geq 0$, $\frac{n(n-1)}{2} \leq \frac{n^2}{2}$

- Therefore, we can take $c = 1/2$ and $N = 0$, to conclude that $T(n) \in O(n^2)$.

Big O Notation: More Examples ..

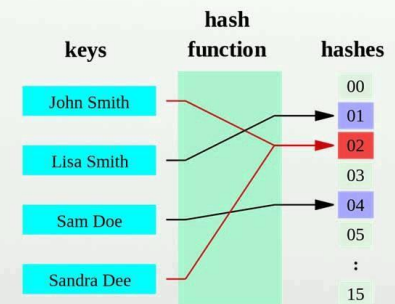
- $n \in O(n^2)$
 - Take $c = 1$ and $N = 1$, then for all n such that $n \geq N$, $n \leq 1 \times n^2$.
- $n^3 \in O(n^2)??$
 - Divide both sides by n^2
 - Then, we can obtain $n \leq c$
 - But it is **impossible** there exists a **constant c** that is large enough than a **variable n**.
 - Therefore, n^3 does **not** belong to $O(n^2)$.

$$n^3 \notin O(n^2)$$

O(1) - Constant Time Examples

- In programming, a most operations are constant . Here are some examples:
 - i. math operations/calculations e.g $x = x + 1$
 - ii. accessing an array via the index e.g $x = S[i]$
 - iii. accessing a hash via the key
 - iv. pushing and popping on a stack
 - v. insertion and removal from a queue
 - vi. returning a value from a function

Hash function



https://en.wikipedia.org/wiki/File:Hash_table_4_1_1_0_0_1_0_LL.svg

Further Reading: [Hash Functions and Types of Hash functions](#)

<https://www.geeksforgeeks.org/hash-functions-and-list-types-of-hash-functions/>

$O(n) \rightarrow$ Linear Time

$O(n^2) \rightarrow$ Quadratic Time

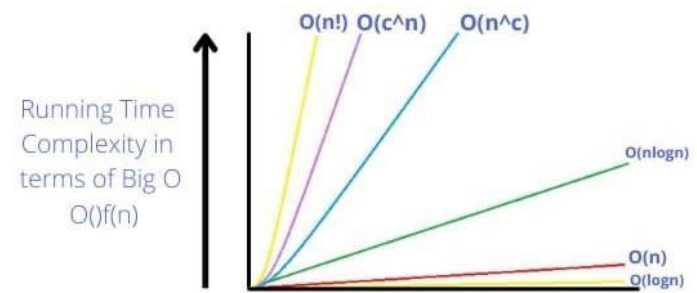
- $O(n)$ means that the run-time increases at the **same pace** as the size of input.
- $O(n^2)$ means that the calculation runs in **quadratic** time, which is the squared size of the input.
- In programming, many of the basic sorting algorithms have a worst-case run time of $O(n^2)$:
 - e.g Bubble Sort, Insertion Sort, Selection Sort

$O(\log n) \rightarrow$ Logarithmic Time

- $O(\log n)$ means that the running time **grows in proportion to the logarithm** of the **input size**.
- this means that the run time **barely increases as you exponentially increase the input.**
- Note: $O(n \log n)$, which is often confused with $O(\log n)$, means that the running time of an algorithm is **linearithmic**, which is a **combination of linear and logarithmic complexity.**
- **Sorting** algorithms that **utilize a divide and conquer** strategy are **linearithmic**, such as the following:
 - **e.g merge sort, timsort, heapsort**
- time complexity, $O(n \log n)$ lands between $O(n^2)$ and $O(n)$.

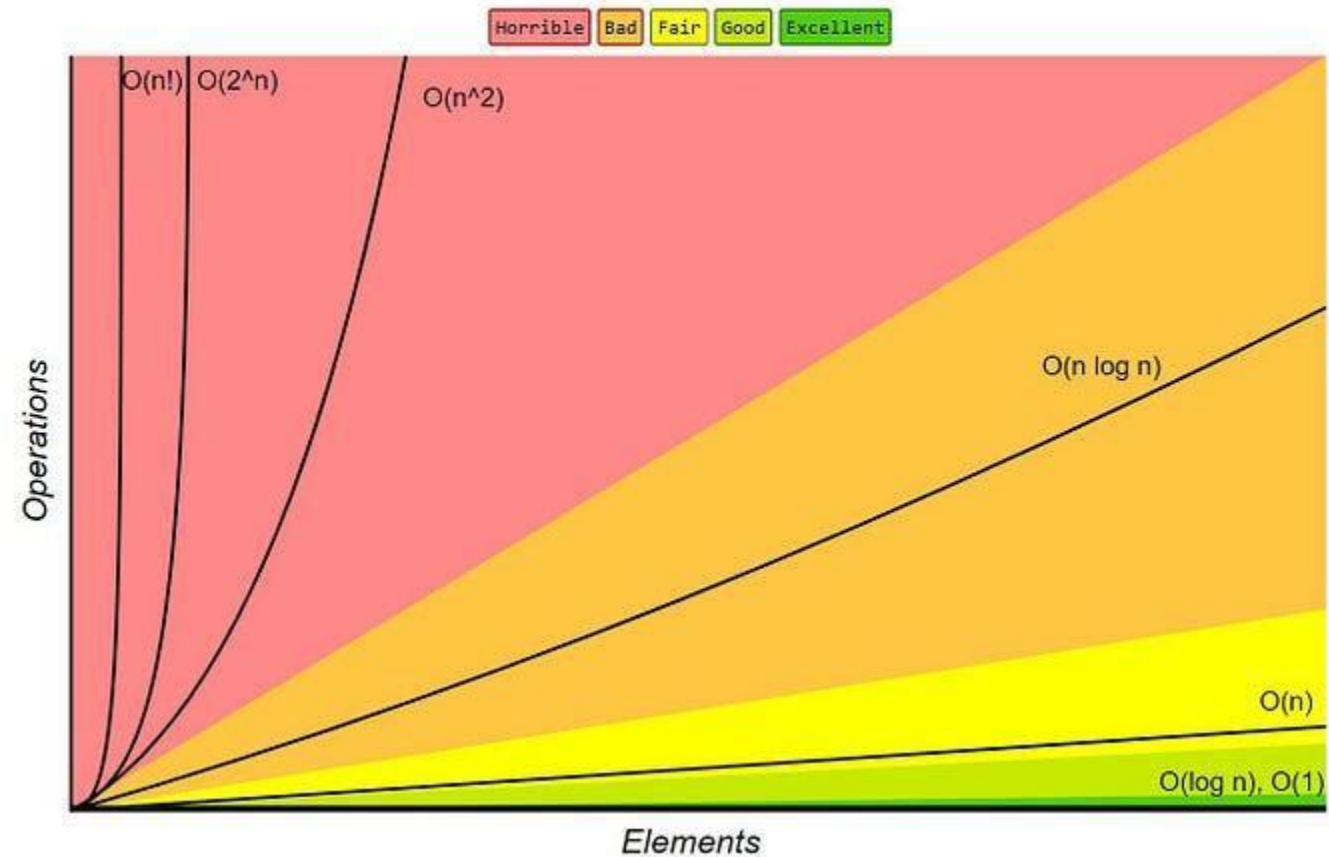
Factorial $O(n!)$

- A good **example** of an algorithm that has factorial time complexity is **the array permutation**.
- In this algorithm, we need to check how many permutations are possible given the array elements.
- For example, if we have **3 elements A, B, and C**, there will be **6 permutations**.
- Let us see how it works:
 - **ABC, BAC, CAB, BCA, CAB, CBA** – Notice that we have **6 possible permutations**, the same as $3!$.
 - If we have 4 elements, we will have **$4!$** which is the same as **24** permutations, if **$5!$ 120** permutations, and so on.



$O(n!)$, $O(c^n)$, $O(n^c)$, Worst
 $O(n \log n)$ - Bad
 $O(n)$ - Fair
 $O(\log n)$ - Good
 $O(1)$ - Best

Big-O Complexity Chart



Common Data Structure Operations

Data Structure	Time Complexity								Space Complexity
	Average				Worst				Worst
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion	
<u>Array</u>	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$
<u>Stack</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$
<u>Queue</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$
<u>Singly-Linked List</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$
<u>Doubly-Linked List</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$
<u>Skip List</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n \log(n))$
<u>Hash Table</u>	N/A	$\theta(1)$	$\theta(1)$	$\theta(1)$	N/A	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$
<u>Binary Search Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$
<u>Cartesian Tree</u>	N/A	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	N/A	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$
<u>B-Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$
<u>Red-Black Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$
<u>Splay Tree</u>	N/A	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	N/A	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$
<u>AVL Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$
<u>KD Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$

Sorting Algorithms	Time Complexity		
	Best Case	Average Case	Worst Case
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$
Bucket Sort	$O(n + k)$	$O(n + k)$	$O(n^2)$

LEGEND

TIME Complexity vs. SPACE Complexity

Good Fair Bad

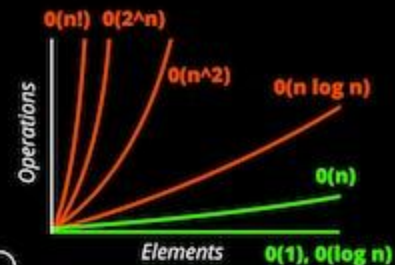
Good Fair Bad



<BIG-O-CHEATSHEET>

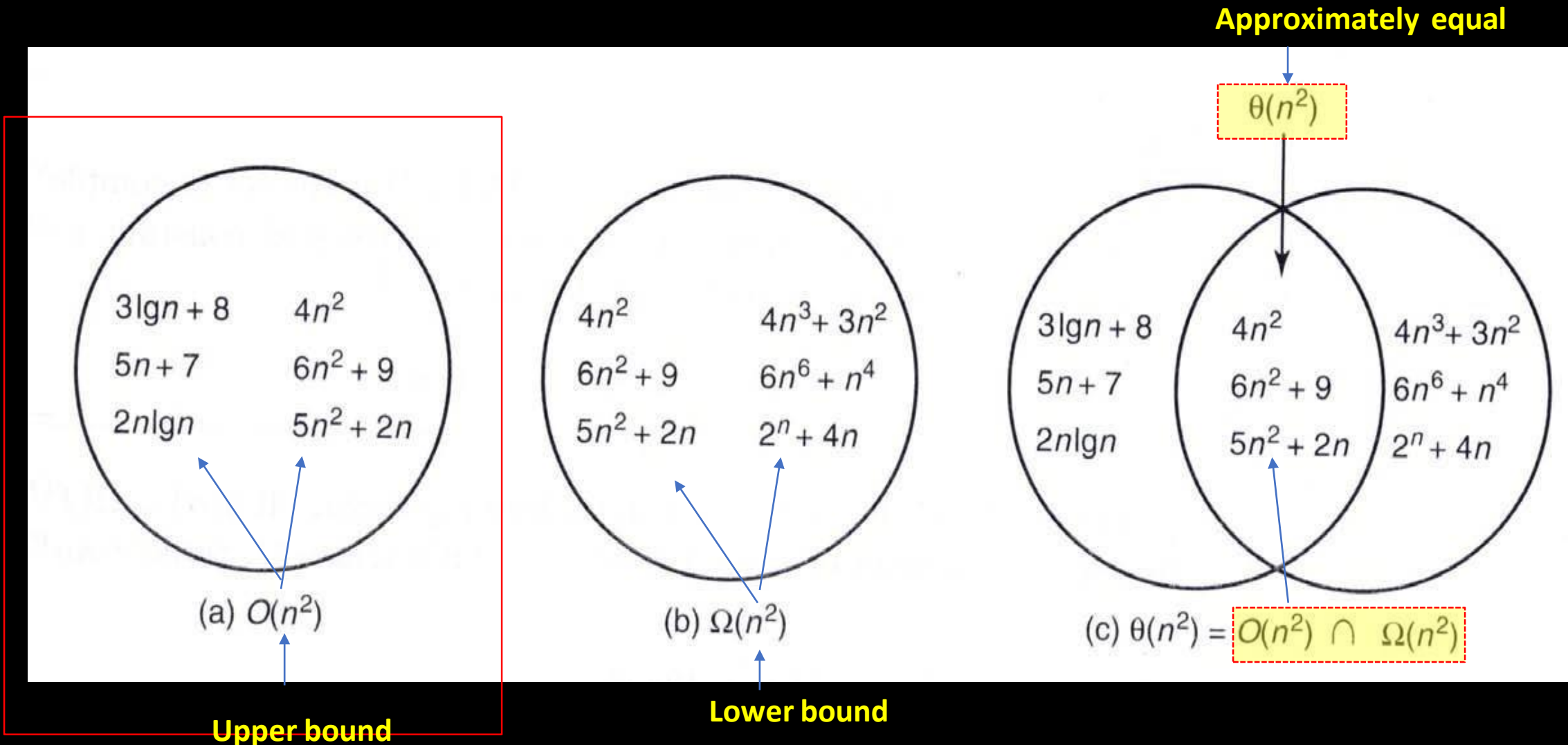


www.bigocheatsheet.com



DATA STRUCTURE Operations										ARRAY SORTING Algorithms								
DATA Structure	TIME Complexity				SPACE Complexity					ARRAY Algorithms	TIME Complexity				SPACE Complexity			
	Average	Average	Average	Average	Worst	Worst	Worst	Worst	Worst		Best	Average	Worst	Worst	Worst	Worst	Worst	
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion										
Array	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	Quicksort	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$\Theta(n^2)$	$\Theta(\log(n))$				
Stack	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$	Mergesort	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$\Theta(n \log(n))$	$\Theta(n)$				
Queue	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$	Timsort	$\Omega(n)$	$\Theta(n \log(n))$	$\Theta(n \log(n))$	$\Theta(1)$				
Singly-Linked List	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$	Heapsort	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$\Theta(n \log(n))$	$\Theta(1)$				
Doubly-Linked List	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$	Bubble Sort	$\Omega(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$				
Skip List	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n \log(n))$	Insertion Sort	$\Omega(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$				
Hash Table	N/A	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	N/A	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	Selection Sort	$\Omega(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$				
Binary Search Tree	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	Tree Sort	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$\Theta(n^2)$	$\Theta(n)$				
Cartesian Tree	N/A	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	N/A	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	Shell Sort	$\Omega(n \log(n))$	$\Theta(n(\log(n))^2)$	$\Theta(n(\log(n))^2)$	$\Theta(1)$				
B-Tree	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(n)$	Bucket Sort	$\Omega(n+k)$	$\Theta(n+k)$	$\Theta(n^2)$	$\Theta(n)$				
Red-Black Tree	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(n)$	Radix Sort	$\Omega(nk)$	$\Theta(nk)$	$\Theta(nk)$	$\Theta(n+k)$				
Splay Tree	N/A	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	N/A	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(n)$	Counting Sort	$\Omega(n+k)$	$\Theta(n+k)$	$\Theta(n+k)$	$\Theta(k)$				
AVL Tree	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(n)$	Cubesort	$\Omega(n)$	$\Theta(n \log(n))$	$\Theta(n \log(n))$	$\Theta(n)$				
KD Tree	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$									

Figure 1.6 The sets $O(n^2)$, $\Omega(n^2)$, $\Theta(n^2)$. Some exemplary members are shown.



Rigorous Definition to Order: Ω (Best Case)

- Definition: (Asymptotic **Lower** Bound)
 - For a given complexity function $f(n)$, $\Omega(f(n))$ is **the set of complexity functions $g(n)$** for which there exists some positive real constant c and some non-negative integer N such that for all $n \geq N$,

$$g(n) \geq c \times f(n)$$

- $g(n) \in \Omega(f(n))$

Ω (Best Case)

- The symbol Ω is the Greek capital letter “omega.”
- If $g(n) \in \Omega(f(n))$, we say that $g(n)$ is omega of $f(n)$. Figure 1.4(b) illustrates Ω .

Example 1.12

We show that $5n^2 \in \Omega(n^2)$. Because, for $n \geq 0$,

$$5n^2 \geq 1 \times n^2,$$

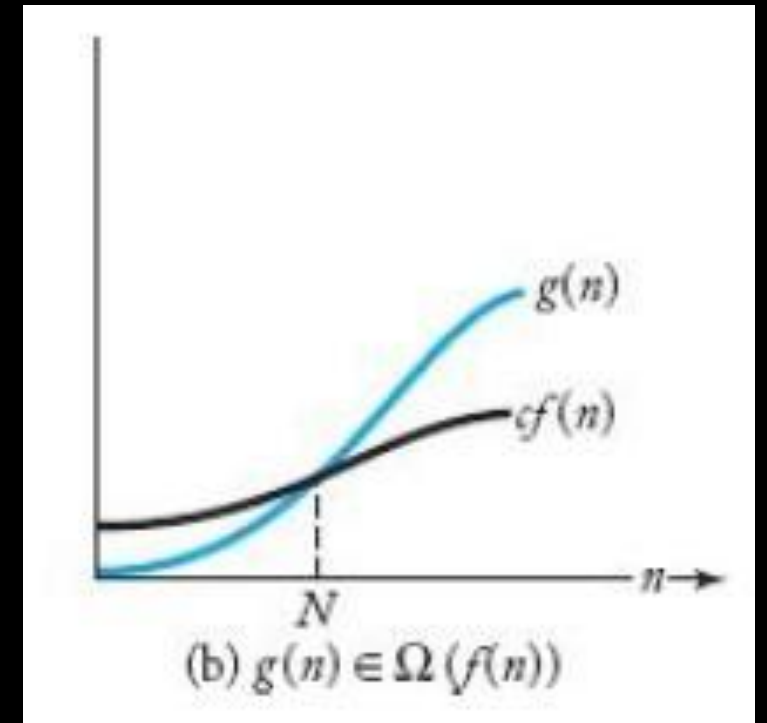
we can take $c = 1$ and $N = 0$ to obtain our result.

Example 1.13

We show that $n^2 + 10n \in \Omega(n^2)$. Because, for $n \geq 0$, $n^2 + 10n \geq n^2$,

$$n^2 + 10n \geq n^2,$$

we can take $c = 1$ and $N = 0$ to obtain our result.



Example 1.14

Consider again the time complexity of Algorithm 1.3 (Exchange Sort). We show that

$$T(n) = \frac{n(n-1)}{2} \in \Omega(n^2).$$

For $n \geq 2$,

$$n-1 \geq \frac{n}{2}.$$

Therefore, for $n \geq 2$,

$$\frac{n(n-1)}{2} \geq \frac{n}{2} \times \frac{n}{2} = \frac{1}{4}n^2,$$

which means we can take $c = 1/4$ and $N = 2$ to obtain our result.

Ω (Best Case)

As is the case for “big O,” there are no unique constants c and N for which the conditions in the definition of Ω hold. We can choose whichever ones make our manipulations easiest.

If a function is in $\Omega(n^2)$, then eventually the function lies above some pure quadratic function on a graph. For the purposes of analysis, this means that eventually it is at least as bad as a pure quadratic function. However, as the following example illustrates, the function need not be a quadratic function.

Ω (Best Case)

Example 1.15

We show that $n^3 \in \Omega(n^2)$. Because, if $n \geq 1$,

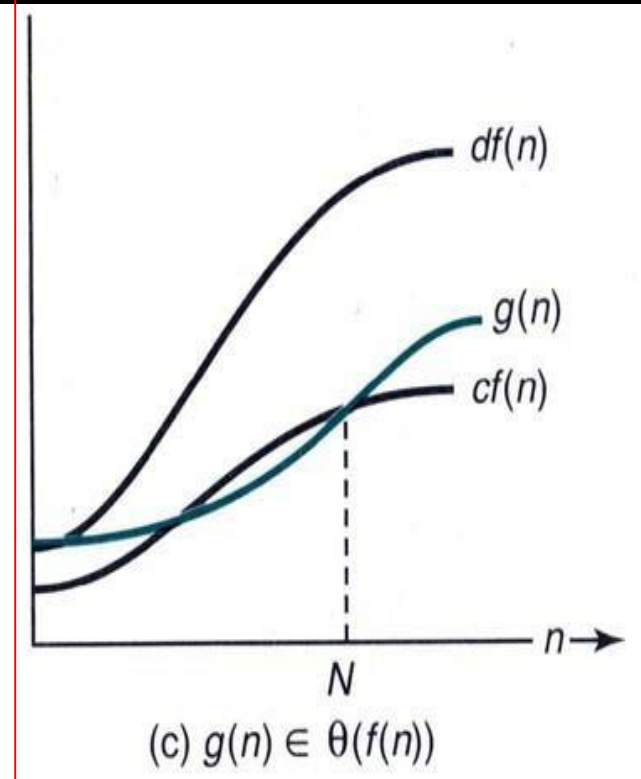
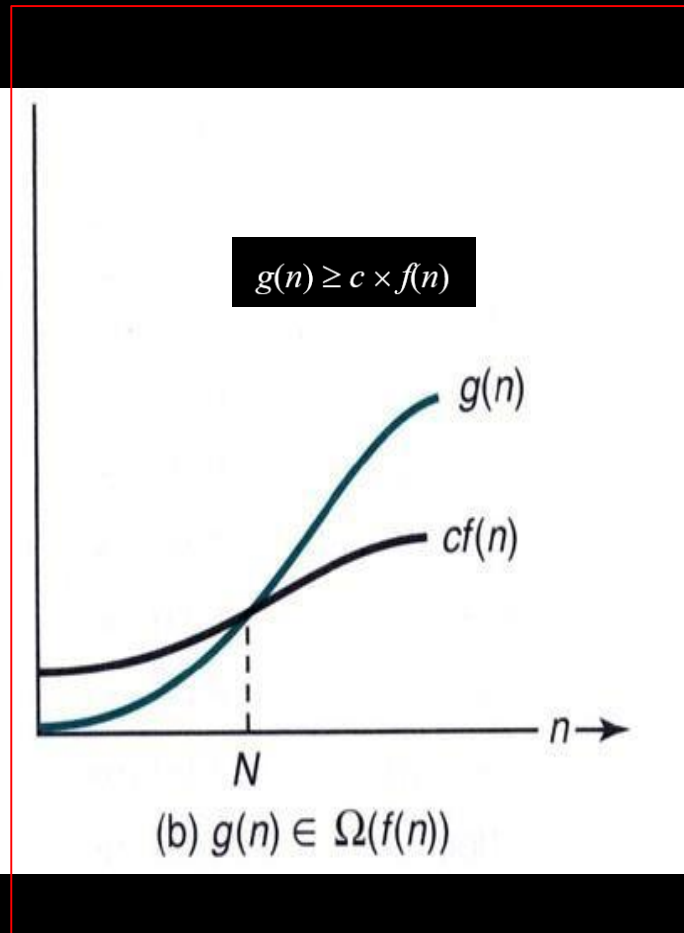
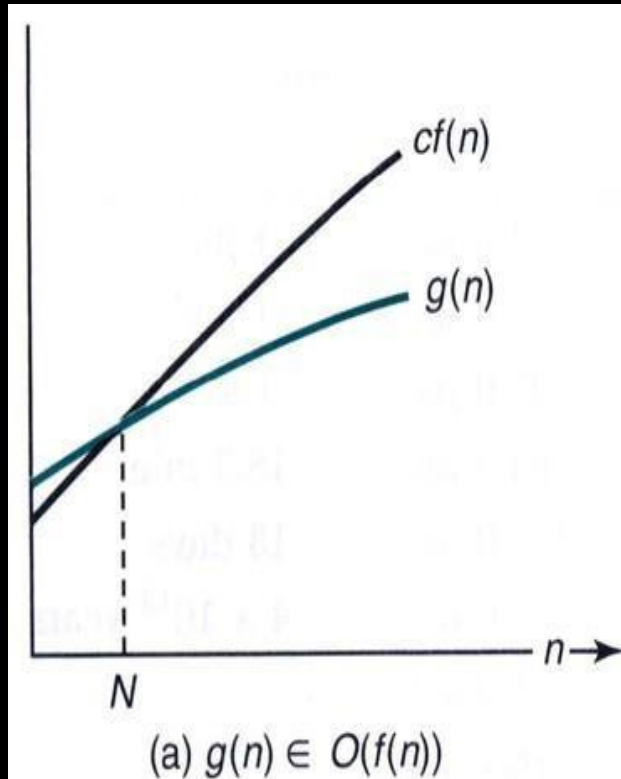
$$n^3 \geq 1 \times n^2,$$

we can take $c = 1$ and $N = 1$ to obtain our result.

Figure 1.6(b) shows some exemplary members of $\Omega(n^2)$

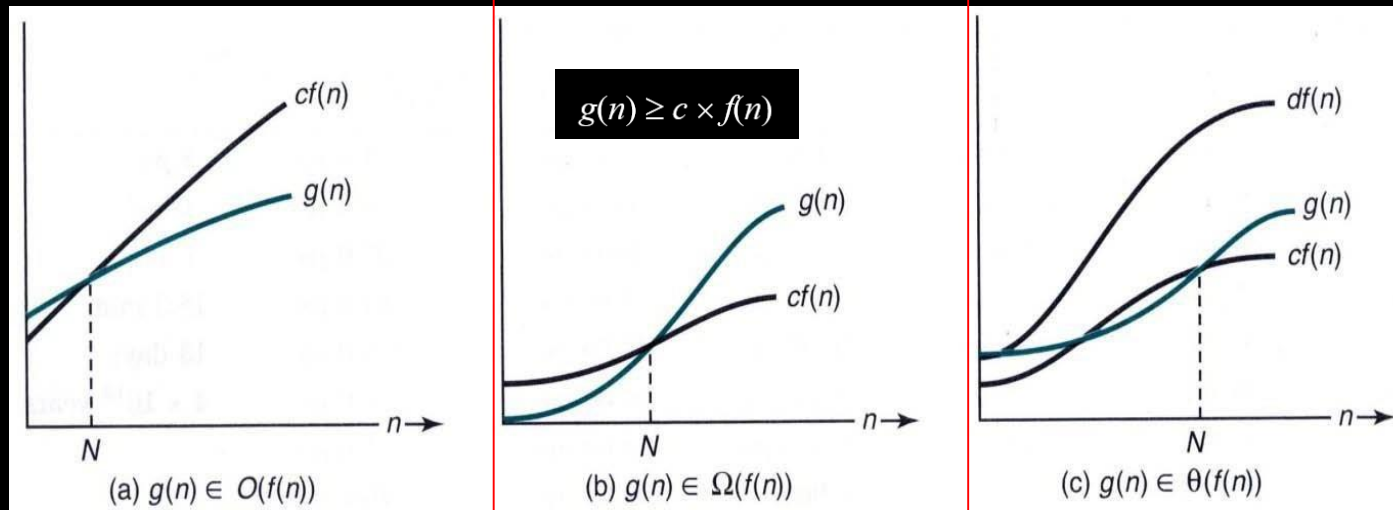
If a function is in both $O(n^2)$ and $\Omega(n^2)$ we can conclude that eventually the function lies beneath some pure quadratic function on a graph and eventually it lies above some pure quadratic function on a graph. That is, eventually it is at least as *good* as some pure quadratic function and eventually it is at least as bad as some pure quadratic function. We can therefore conclude that its growth is similar to that of a pure quadratic function. This is precisely the result we want for our rigorous notion of order. We have the following definition.

Illustrating “**big O** (Worst Case)”, **Ω** (Best Case), and **Θ** (Average Case)



Ω (Best Case) Notation: Definition

- Meaning of $g(n) \in \Omega(f(n))$
 - Although $g(n)$ starts out below $cf(n)$ in the figure, **eventually** it goes **above** $cf(n)$ and stays there.
 - If $g(n)$ is the time complexity for some algorithm, eventually the running time of the algorithm will be at least as bad as $f(n)$
 - $f(n)$ is called as an asymptotic **lower** bound (of what?)
 - (i.e. $g(n)$ **cannot** run **faster** than $f(n)$, **eventually**)



Ω (Best Case) Notation: Example

- Meaning of $n^2+10n \in \Omega(n^2)$
 - Take $c = 1$ and $N = 0$.
 - For all integer $n \geq 0$, it holds that $n^2+10n \geq n^2$
 - Therefore, n^2 is an asymptotic lower bound for the time complexity function of n^2+10n .
 - (i.e., n^2+10n belongs to $\Omega(n^2)$)
- $5n^2 \in \Omega(n^2)$
 - Take $c = 1$ and $N = 0$.
 - For all integer $n \geq 0$, it holds that $5n^2 \geq 1 \times n^2$
 - Therefore, n^2 is an asymptotic lower bound for the time complexity function of $5n^2$.

Ω (Best Case) Notation: Examples

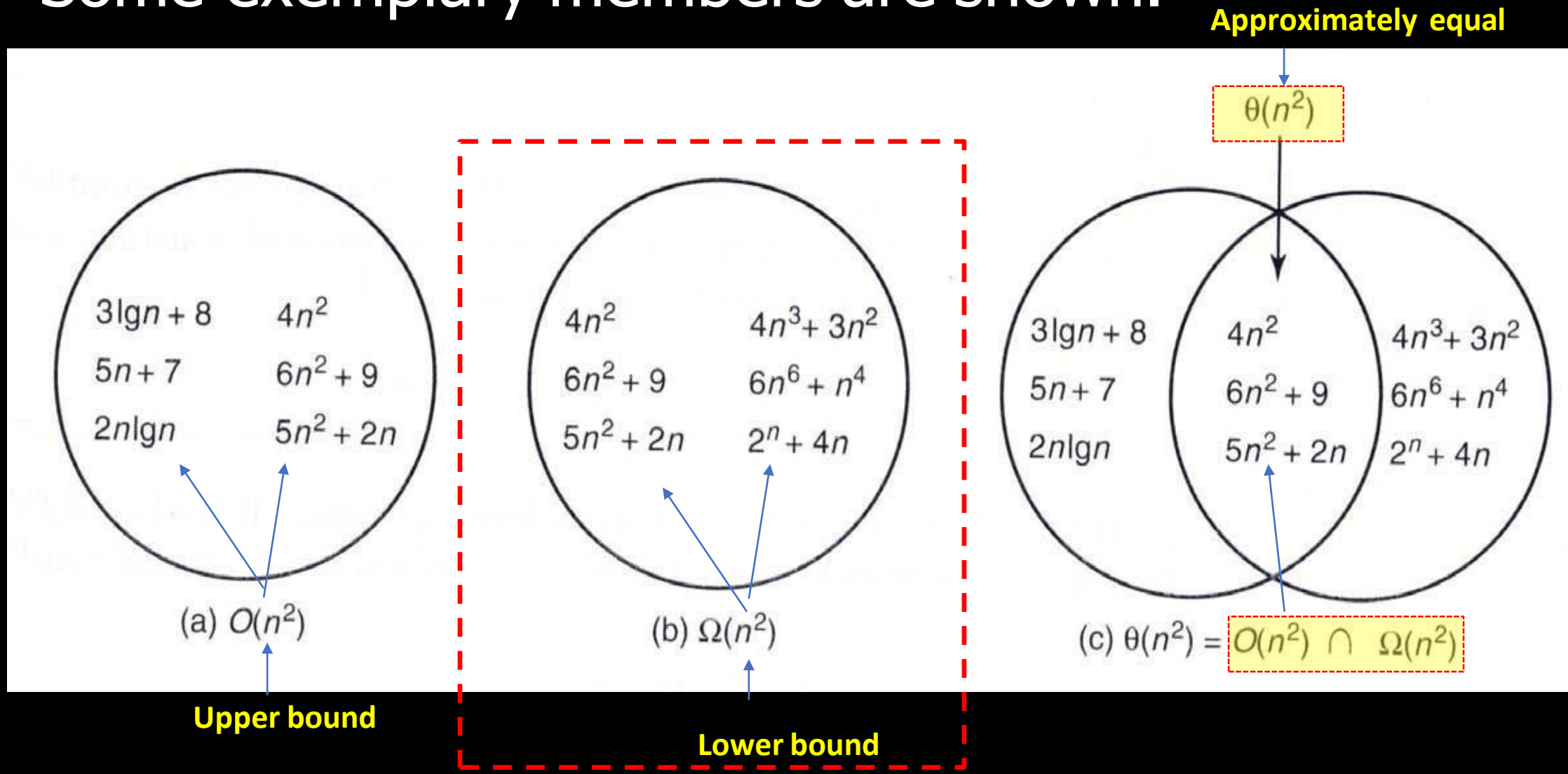
- $T(n) = \frac{n(n-1)}{2}$
 - Because, for $n \geq 2$, $n - 1 \geq n/2$, so it holds that $\frac{n(n-1)}{2} \geq \frac{n}{2} \times \frac{n}{2} = \frac{1}{4}n^2$
 - Therefore, we can take $c = 1/4$ and $N = 2$, to conclude that $T(n) \in \Omega(n^2)$.
- $n^3 \in \Omega(n^2)$
 - Because, for $n \geq 1$, it holds that $n^3 \geq 1 \times n^2$
 - Therefore, we can take $c = 1$ and $N = 1$, to conclude that $n^3 \in \Omega(n^2)$

Ω (Best Case) Notation: Example

- $n \in \Omega(n^2)$??
 - Proof by **contradiction**.
 - **Suppose** it is true that $n \in \Omega(n^2)$.
 - Then, for all integer $n \geq N$, there must exist some positive real number $c > 0$, and **non-negative integer** N .
 - Let us **divide both sides by** cn .
 - Then, we will get $1/c \geq n$, which is **impossible**.
 - Therefore, n does **not** belong to $\Omega(n^2)$.

$$n \notin \Omega(n^2)$$

Figure 1.6 The sets $O(n^2)$, $\Omega(n^2)$, $\Theta(n^2)$.
Some exemplary members are shown.



Average Case Θ (Theeta)

Definition

For a given complexity function $f(n)$,

$$\Theta(f(n)) = O(f(n)) \cap \Omega(f(n)).$$

This means that $\Theta(f(n))$ is the set of complexity functions $g(n)$ for which there exists some positive real constants c and d and some nonnegative integer N such that, for all $n \geq N$,

$$c \times f(n) \leq g(n) \leq d \times f(n).$$

Example 1.16

Consider once more the time complexity of Algorithm 1.3. Examples 1.8 and 1.14 together establish that

$$T(n) = \frac{n(n-1)}{2} \quad \text{is in both} \quad O(n^2) \quad \text{and} \quad \Omega(n^2).$$

This means that $T(n) \in O(n^2) \cap \Omega(n^2) = \Theta(n^2)$

Figure 1.6(c) depicts that $\Theta(n^2)$ is the intersection of $O(n^2)$ and $\Omega(n^2)$, whereas

Rigorous Definition to Order: Θ

- Definition: (Asymptotic **Tight** Bound)
 - For a given complexity function $f(n)$, $\Theta(f(n))$ is **the set of complexity functions $g(n)$ for which there exists some positive real constants c and d and some non-negative integer N such that for all $n \geq N$,**

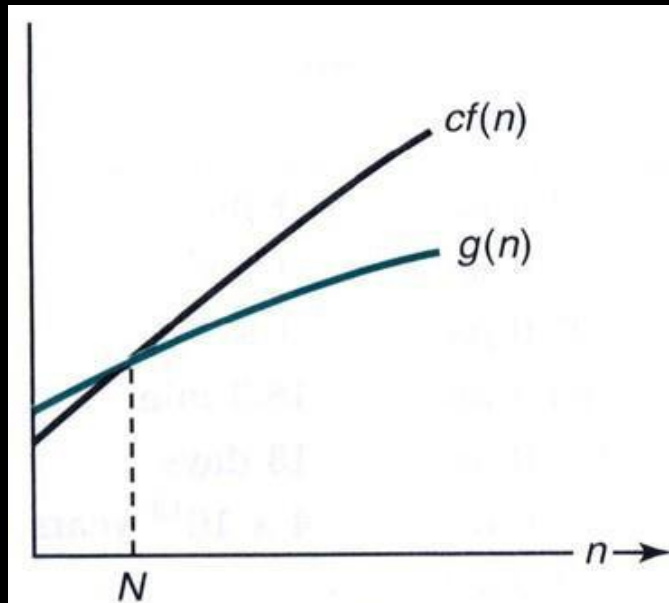
$$c \times f(n) \leq g(n) \leq d \times f(n)$$

- $g(n) \in \Theta(f(n))$, we say that $g(n)$ is order of $f(n)$.

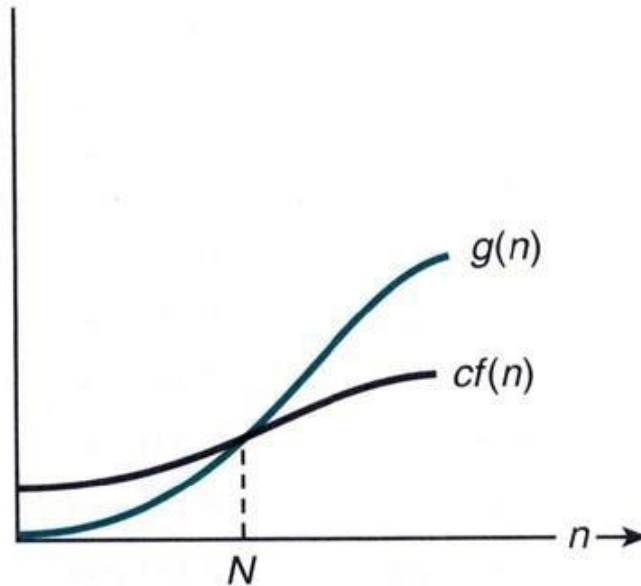
- Example:

$$T(n) = \frac{n(n-1)}{2} \quad T(n) \in \Theta(n^2)$$

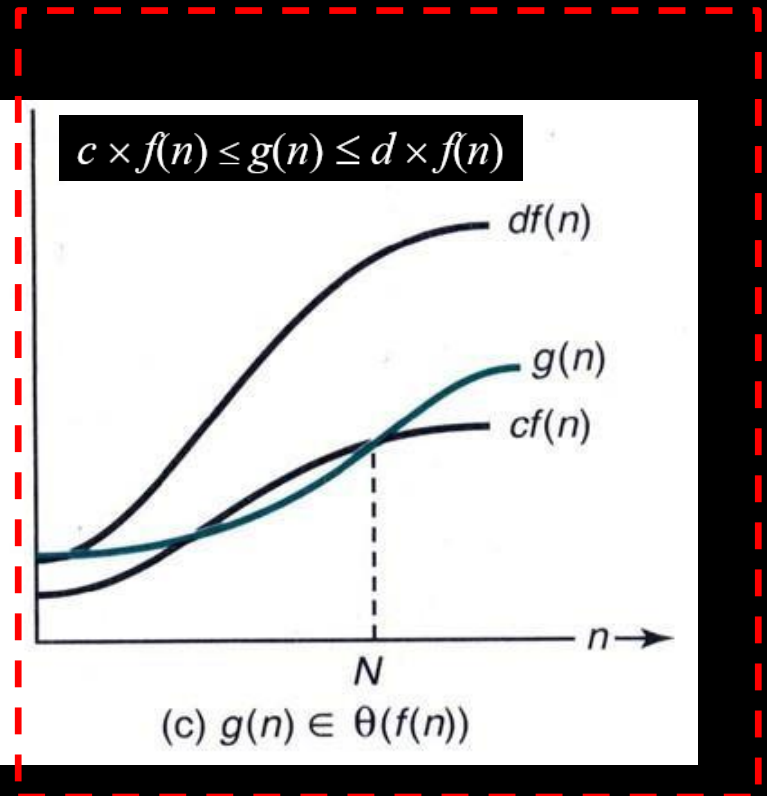
Illustrating “big O”, Ω , and Θ



(a) $g(n) \in O(f(n))$



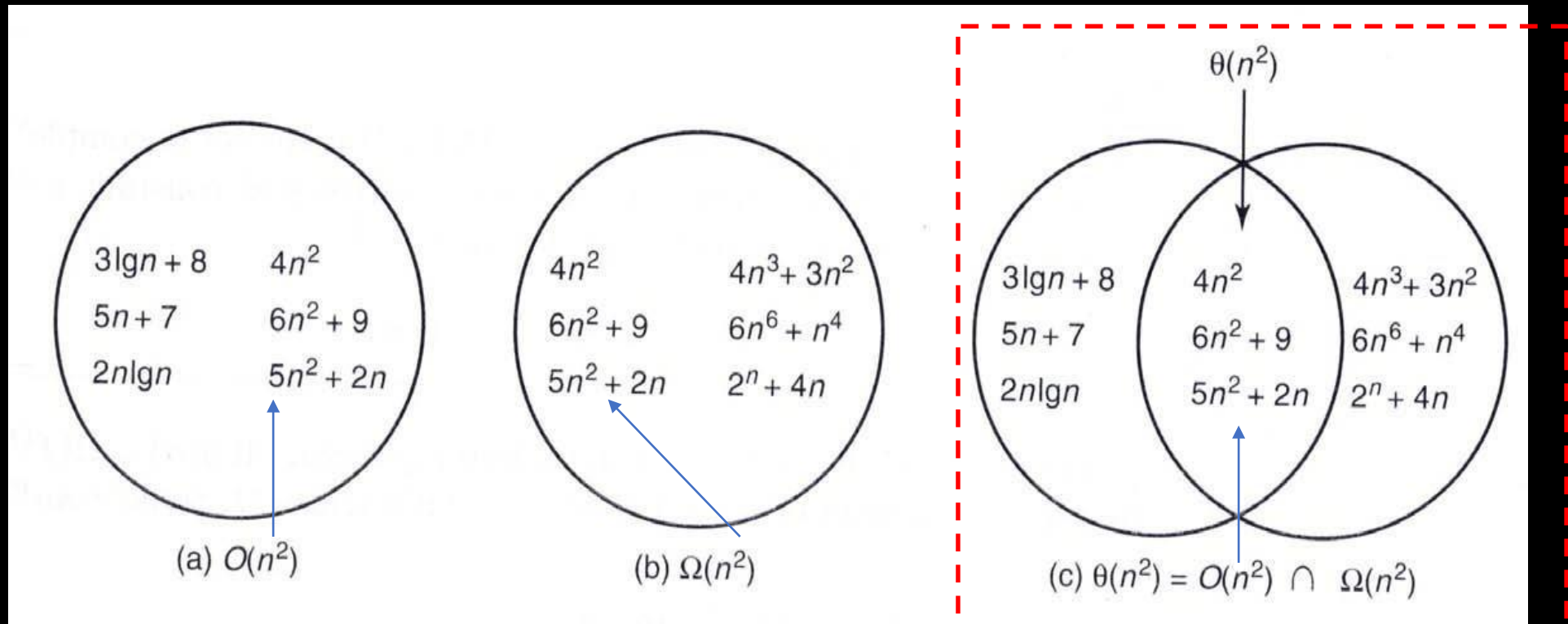
(b) $g(n) \in \Omega(f(n))$



(c) $g(n) \in \Theta(f(n))$

Figure 1.6 The sets $O(n^2)$, $\Omega(n^2)$, $\Theta(n^2)$.
Some exemplary members are shown.

Approximately equal



Upper bound

Lower bound

Rigorous Definition to Order: Small o

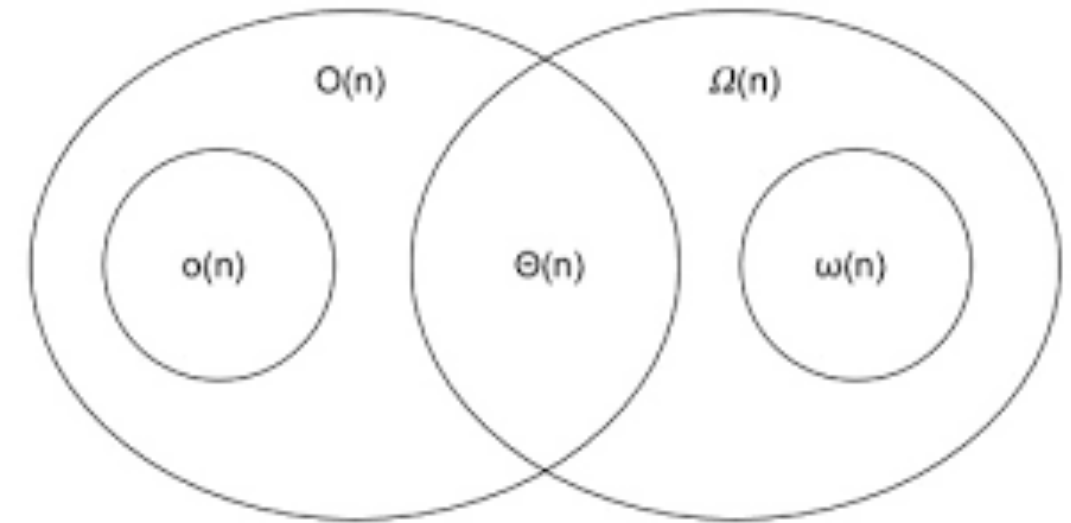
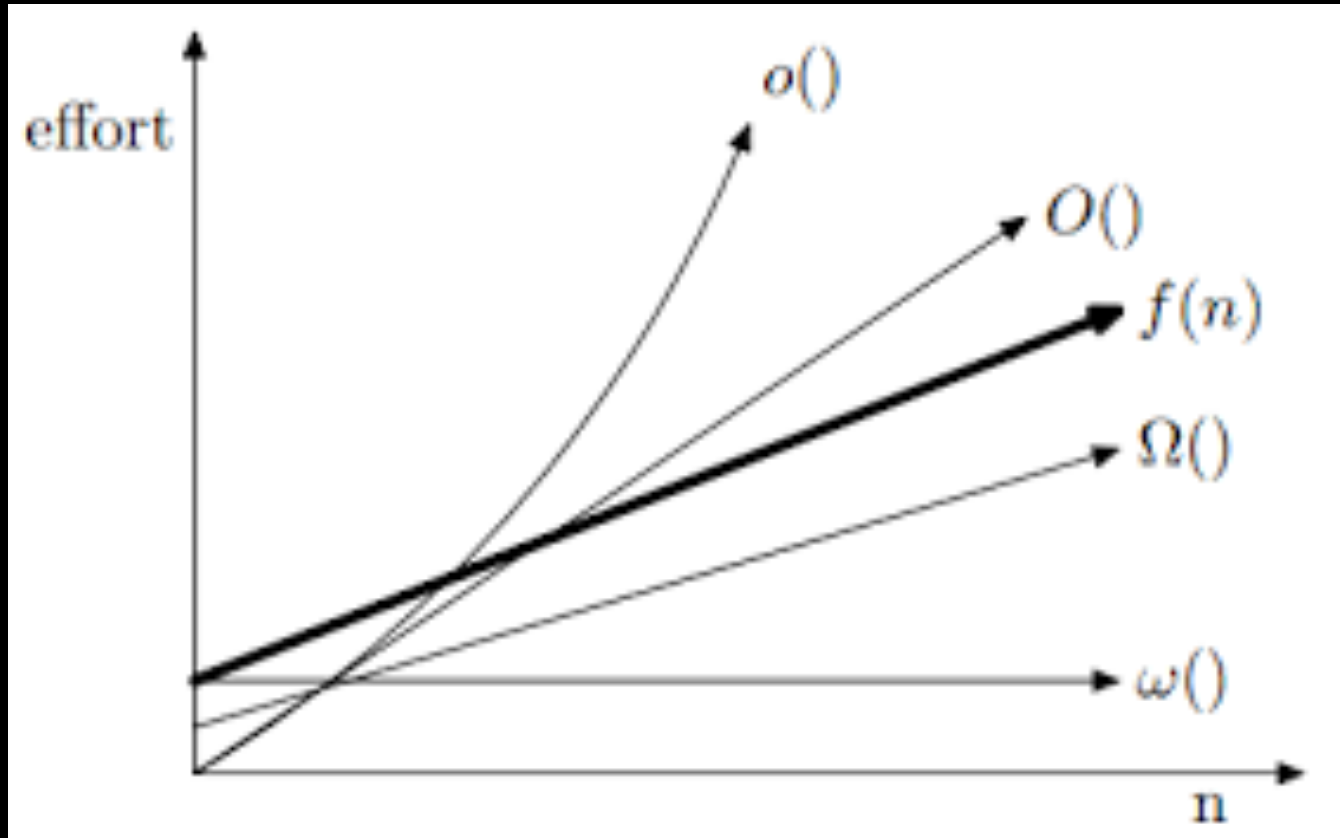
■ Definition:

- For a given complexity function $f(n)$, $o(f(n))$ is **the set of complexity functions $g(n)$** satisfying the following: For **every positive real constant c** there **exists** a non-negative integer N such that **for all $n \geq N$,**

$$g(n) \leq c \times f(n)$$

- $g(n) \in o(f(n))$

Big O vs Small o and Big Omega vs small omega



Big O vs Small o

- Both describe upper bounds, although somewhat counter-intuitively, **Little-o** is the **stronger statement**.
- There is a much **larger gap** between the **growth rates of f and g** if $f \in o(g)$ than if $f \in O(g)$.

Big O Notation (O)

- The function's growth rate is "**less than or equal to**" a constant multiple of another function. It provides an upper bound, meaning the function will not grow faster than the specified limit.
- Analogous to the " $\leq$ " (less than or equal to) operator.

Example: $2n^2 + 5$ is $O(n^2)$.

Small o Notation (o)

- The function grows **strictly slower** than another function. As the variable grows, the ratio of the two functions approaches zero.
- Analogous to the "<" (strictly less than) operator.

Example: n is $o(n^2)$ because

$$\lim_{n \rightarrow \infty} \frac{n}{n^2} = 0.$$

Big O vs. Small o

■ Difference

- **Big O**: For a given complexity function $f(n)$, $O(f(n))$ is the set of complexity functions $g(n)$ for which there exists **some** positive real constant c and **some** non-negative integer N such that for all $n \geq N$
- **Small o**: For a given complexity function $f(n)$, $o(f(n))$ is the set of complexity functions $g(n)$ satisfying the following: For **every** positive real constant c **there exists** a non-negative integer N such that for all $n \geq N$,

$$g(n) \leq c \times f(n)$$

- If $g(n) \in o(f(n))$, $g(n)$ is eventually **much better** than $f(n)$.
- Big O and small o both asymptotic **notations that specify** upper-bounds for functions and running times of algorithms.
- ****However, the difference is that big-O may be asymptotically tight while little-o makes sure that the upper bound is not asymptotically tight**

Small o Notation: Example

- $n \in o(n^2)$
 - Suppose $c > 0$. We need to find an N such that, for $n \geq N$, $n \leq cn^2$.
 - If we divide both sides by cn ,
 - Then, we get $1/c \leq n$
 - Therefore, it sufficient to choose any $N \geq 1/c$.
 - For example, if $c=0.00001$, we must take N equal to at least 100,000.
 - That is, for $N \geq 100,000$, $n \leq 0.00001n^2$.

Small o Notation: Example2

- $n \in o(5n)$?
 - Proof by contradiction.
 - Let $c = 1/6$. If $n \in o(5n)$, then there must exist **some** N such that, for $n \geq N$,
$$n \leq \frac{1}{6} \times 5n = \frac{5}{6} n$$
 - But it is **impossible**.
 - This contradiction proves that n is not in $o(5n)$.

Summary

1. best-case scenario of a time complexity can be described as Omega Big- Ω notation
2. average-case scenario of a time complexity can be described as Theta Big- Θ notation
3. worst-case scenario of time complexity and the most important that is the Big-O notation
4. $O(1)$: **constant** time. Examples of **accessing a number from an array**. Calculating numbers...
5. $O(\log n)$: **Logarithmic** time. **Uses the divide and conquer strategy**.
 - The binary search and [tree binary search](#) are good examples of algorithms that have this time complexity.
 - An **approximate real-world example** would be to search a word in the **dictionary**.



Summary ctd..

6. $O(n)$ – **Linear time**: When we traverse the whole array once, we have the $O(n)$ time complexity.
 - When we store information in an array of n we will also have the $O(n)$ for space complexity.
 - A real-world example could be reading a book.
7. $O(N \log N)$ – **Log-linear/ linearithmic** : The Merge sort, Quick Sort, Tim Sort, and Heap Sort are algorithms that have **log-linear complexity**.
 - Those algorithms use the divide and conquer strategy making it more effective than $O(n^2)$.

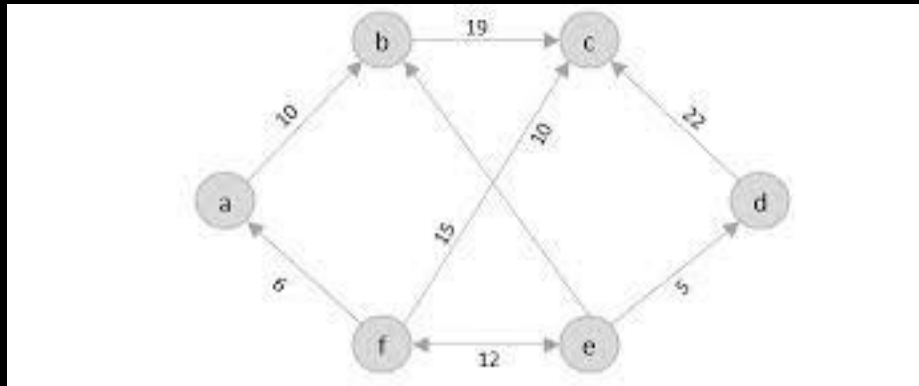
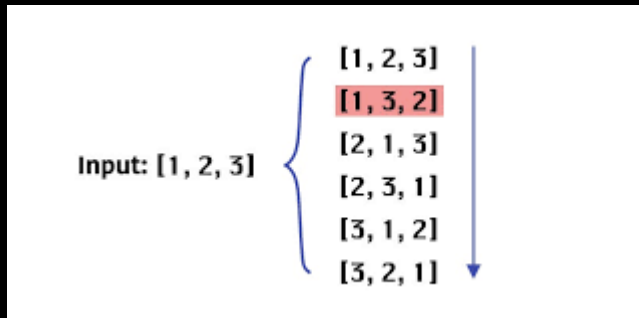
Summary ctd..

8. $O(N^2)$ – **Quadratic**: The Bubble Sort, Insertion Sort, and Select Sort are some of the algorithms that have the **quadratic complexity**.
 - If there are **two nested loops traversing the whole array each time**, then we have $O(n^2)$ of time complexity.
9. $O(N^3)$ – **Cubic**: When we have **3 nested loops and we traverse the whole array on those loops** we will have the cubic time complexity.
10. $O(c^n)$ – **Exponential**: The exponential complexity grows very quickly.
 - A good example of this time complexity is **when someone try to break a password**.
 - Suppose it is a password that supports **only numbers(0-9)** and has **4 digits**. That equivalates to $10^4 = 10000$ possible combinations.
 - The **greater the exponent number the faster the number grows**.

Summary ctd..

11. $O(n!)$ – **Factorial**: The factorial of $3!$ is the same as $1 * 2 * 3 = 6$. It grows in a similar way to exponential complexity.

- The algorithms that have this time complexity are **array permutation** and **traveling salesman**.



Refer <https://www.bigocheatsheet.com/>
<https://javachallengers.com/big-o-notation-explanation/>

Questions?



Problem Solving Strategies and Computational Approaches SCS1304

Handout 4 : Time Complexity & Asymptotic notation

Prasad Wimalaratne PhD(Salford),SMIEEE

Overview

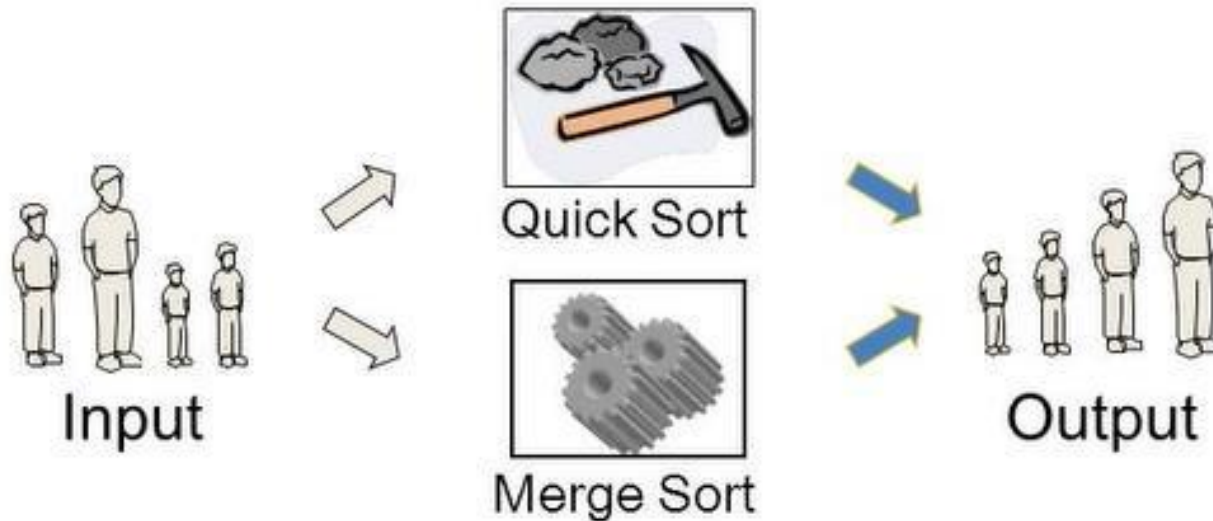
- Analysis of Algorithms
 - Order of Algorithms
 - Practical Considerations
- Worst-case time complexity
- Best-case time complexity
- Average-case time complexity

[Chapter 1] (part2)
Algorithms :
Efficiency, Analysis,
And Order

Analysis of Algorithms

An algorithm is a step-by-step procedure for solving a problem in a finite amount of time.

How to evaluate algorithms?



- Which one is better?
- What are the criteria?

Analysis of Algorithms

Analysis involves evaluating algorithms to understand their **efficiency, performance, and behavior**:

1. **Correctness**: Ensuring that an algorithm produces the correct output for all possible valid inputs.
2. **Complexity Analysis**: Determining the time and space complexity of an algorithm. This helps in predicting and comparing the **efficiency** of different algorithms.
3. **Empirical Analysis**: Testing algorithms on real data sets to measure their actual performance in practice. This complements theoretical complexity analysis.

Order of Algorithms

- “Order” categorizes algorithms based on their efficiency and complexity:
 1. **Comparison of Sorting**: Algorithms like Merge Sort, Quick Sort, and Heap Sort are categorized by their time complexity for sorting elements in arrays or lists.
 2. **Searching Algorithms**: Techniques like Binary Search and Linear Search are evaluated based on their time complexity for finding elements in data structures.
 3. **Dynamic Programming**: Algorithms that break down complex problems into smaller subproblems and use memoization or tabulation to optimize solutions.

Practical Considerations

1. **Algorithm Selection:** Choosing the right algorithm based on problem requirements, input size, and desired performance characteristics.
2. **Optimization Techniques:** Strategies such as pruning, memoization, and parallelism to improve algorithm efficiency and reduce time or space complexity.
3. **Algorithm Design Paradigms:** Understanding different approaches like divide and conquer, greedy algorithms, and dynamic programming for solving diverse computational problems efficiently.

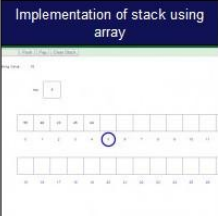
(to be covered in upcoming lessons)

Algorithm Animations and Visualizations

- [collection of computer science algorithm animations and visualizations](https://algoanim.ide.sk/)
- Refer the following URL for visualizing commonly used algorithms
- <https://algoanim.ide.sk/>

TOP 4 ★ FUNDAMENTALS 1 ★ show all (8) animation

Implementation of stack using array



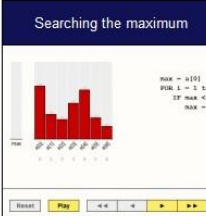
JavaScript animation

Searching the minimum



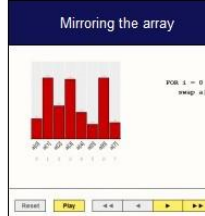
JavaScript animation

Searching the maximum



JavaScript animation

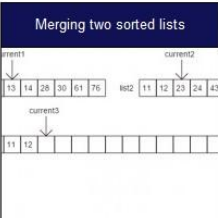
Mirroring the array



JavaScript animation

TOP 4 ★ FUNDAMENTALS 2 ★ show all (5) animation

Merging two sorted lists



JavaScript animation

Binary search



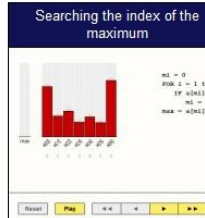
JavaScript animation

Searching the index of the minimum



JavaScript animation

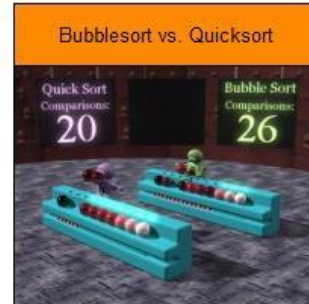
Searching the index of the maximum



JavaScript animation

TOP 4 ★ BUBBLESORT ★ show all (5) animation

Bubblesort vs. Quicksort



Video

Bubblesort with LEGO



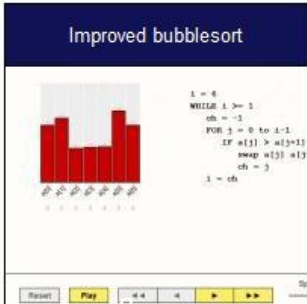
Video

Bubblesort



JavaScript animation

Improved bubblesort



JavaScript animation

Basic Operation in Time Complexity

- Definition: A basic operation is the **dominant operation** whose number of executions is counted to determine an algorithm's time complexity.
- In time complexity analysis, a basic operation is **the operation that is executed most frequently** and whose count is used to estimate the algorithm's running time.

Examples:

- i. Searching in an array : comparison ($A[i] == \text{key}$)
- ii. Sorting algorithms : comparison ($A[i] > A[j]$)
- iii. Matrix multiplication : multiplication ($A[i][k] * B[k][j]$)
- iv. Summing array elements : addition ($\text{sum} = \text{sum} + A[i]$)

Example:

for i = 1 to n

*sum = **sum + A[i]**;*

Basic operation=> addition (**sum + A[i]**)

It executes **n times**, so the time complexity is **O(n)**.

Analysis of Algorithms

- In general, the **running time** of an algorithm **increases with the size of the input**, and the total running time is **roughly proportional** to how many times some basic operation (such as a comparison instruction) is carried out.
- We therefore analyze the algorithm's efficiency by determining the number of times some basic operation is carried out as a function of the size of the input.
- After determining the input size, we pick some instruction or group of instructions such that the **total work done by the algorithm** is roughly **proportional** to the number of times **this instruction or group of instructions** is done.
- We call this instruction or group of instructions the **basic operation** in the algorithm.
 - e.g how many times x is compared with elements of the array in earlier examples ?

Analysis of Algorithms ctd..

- Although we **do not want to consider the details of how an algorithm is implemented**, we will ordinarily assume that the basic operation is implemented **as efficiently as possible**.
- **best, worst, and average cases** of a given algorithm express what the **resource usage** is at **least, at most and on average**, respectively.
- Usually the resource being considered is running time, i.e. time complexity, but could also be memory or some other resource.

Worst-case time complexity analysis

- The first is to consider the maximum number of times the basic operation is done.
- For a given algorithm, $W(n)$ is defined as the maximum number of times the algorithm will ever do its basic operation for an input size of n .
- $W(n)$ is called the worst-case time complexity of the algorithm, and the determination of $W(n)$ is called a worst-case time complexity analysis.

Worst-Case Time Complexity (Sequential Search)

- Basic operation: the comparison of an item in the array with x .
- Input size: n , the number of items in the array.
- The basic operation is done at most n times, which is the case if x is the last item in the array or if x is not in the array. Therefore,

$$W(n) = n.$$

Algorithm 1.1

Sequential Search

Problem: Is the key x in the array S of n keys?

Inputs (parameters): positive integer n , array of keys S indexed from 1 to n , and a key x .

Outputs: $location$, the location of x in S (0 if x is not in S).

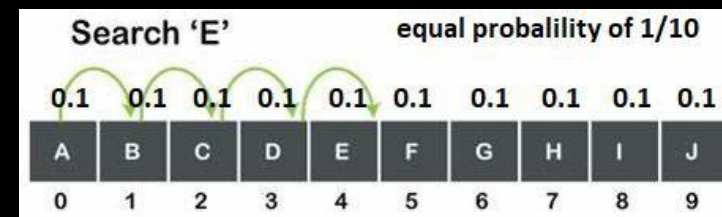
```
void seqsearch (int n,
                const keytype S[ ],
                keytype x,
                index& location)
{
    location = 1;
    while (location <= n && S[location] != x)
        location++;
    if (location > n)
        location = 0;
}
```

Worst-Case Time Complexity (Sequential Search)

- Although the worst-case analysis informs us of **the absolute maximum amount of time consumed**, in some cases we may be more interested in knowing how the algorithm performs **on the average**.
- For a given algorithm, $A(n)$ is defined as the average (expected value) of the number of times the algorithm does the basic operation for an input size of n .
- $A(n)$ is called the average-case time complexity of the algorithm, and the determination of $A(n)$ is called an **average-case time complexity analysis**.

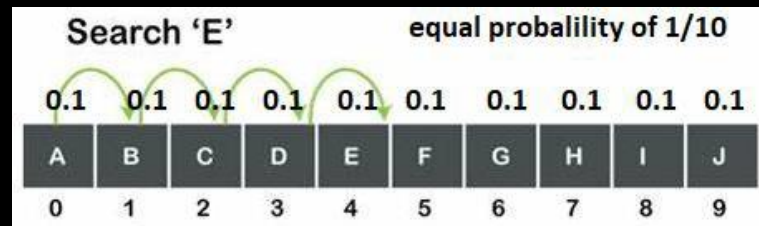
Average -Case Time Complexity (Sequential Search)

- To compute $A(n)$, we need to assign probabilities to all possible inputs of size n .
- It is important to assign probabilities based on all available information.
- For example, our next analysis will be an average-case analysis of Sequential Search
- Assumption: We will assume that if x is in the array, it is equally likely to be in any of the array slots.
- If we know only that x may be somewhere in the array, our information gives us no reason to prefer one array slot over another.



Average -Case Time Complexity (Sequential Search) ctd.

- Therefore, it is reasonable to **assign equal probabilities** to all array slots.
- This means that we are **determining the average search time** when we search for all items the same number of times.



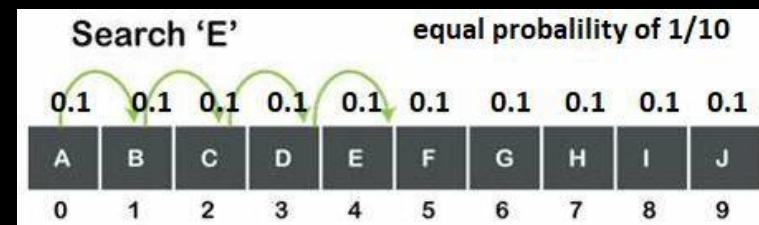
- **Note:** If we have information indicating that the inputs will NOT arrive according to this distribution, we should NOT use this distribution in our analysis.

Average -Case Time Complexity (Sequential Search) ctd..

- For example, if the array contains first names and we are searching for names that have been chosen at random from all people in a country, an array slot containing the common names will probably be searched more often than one containing the uncommon names
- We should NOT ignore this information and assume that all slots are equally likely.
- it is usually harder to analyze the average case than it is to analyze the worst case.

Average-Case Time Complexity (Sequential Search)

- Basic operation: the comparison of an item in the array with x .
- Input size: n , the number of items in the array.
- We first analyze the case in which it is **known** that **x is in S** , **where the items in S are all distinct**, and where we have no reason to believe that x is more likely to be in one array slot than it is to be in another.
- Based on this information, for $1 \leq n$, the probability **that x is in the k^{th} array slot is $1/n$.**



Search 'E' equal probability of 1/10

0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1
A	B	C	D	E	F	G	H	I	J
0	1	2	3	4	5	6	7	8	9

Average-Case Time Complexity (Sequential Search) ctd..

- If x is in the k^{th} array slot, the number of times the basic operation is done to locate x (and, therefore, to exit the loop) is k .
- This means that the average time complexity is given by

$$A(n) = \sum_{k=1}^n \left(k \times \frac{1}{n} \right) = \frac{1}{n} \times \sum_{k=1}^n k = \frac{1}{n} \times \frac{n(n+1)}{2} = \frac{n+1}{2}.$$

- As we would expect, on the average, about half the array is searched.

Average-Case Time Complexity (Sequential Search) ctd..

- Next we analyze the **case in which x may not be in the array.**
- To analyze this case we must **assign some probability p to the event that x is in the array.**
- **If x is in the array,** we will again **assume that it is equally likely to be in any of the slots** from 1 to n.
- The probability **that x is in the k^{th} slot is then p/n ,** and the **probability that it is not in the array is $1 - p$.**

Average-Case Time Complexity (Sequential Search) ctd..

- Recall(Algo 1.2) that there are k passes through the loop if x is found in the kth slot, and n passes through the loop if x is not in the array.
- The average time complexity is therefore given by


$$\begin{aligned} A(n) &= \sum_{k=1}^n \left(k \times \frac{p}{n} \right) + n(1-p) \\ &= \frac{p}{n} \times \frac{n(n+1)}{2} + n(1-p) = n \left(1 - \frac{p}{2} \right) + \frac{p}{2}. \end{aligned}$$

- The last step in this triple equality is derived with algebraic manipulations. If p = 1, A(n) = (n + 1)/2, as before, whereas if p = 1/2, A(n) = 3n/4 + 1/4.
- This means that about 3/4 of the array is searched on the average.

Average-Case Time Complexity (Sequential Search) ctd..

- Caution1: An **average** can be called “**typical**” **only if** the **actual** cases **do not deviate** much from the **average** (that is, only if the **standard deviation is small**).

Best-case time complexity analysis.

- A final type of time complexity analysis is the determination of the **smallest number of times the basic operation is done**.
- For a given algorithm, $B(n)$ is defined as the minimum number of times the algorithm will ever do its basic operation for an input size of n .
- So $B(n)$ is called the **best-case time** complexity of the algorithm, and the **determination of $B(n)$** is called a **best-case time complexity analysis**.

Best-Case Time Complexity (Sequential Search)

- Basic operation: the comparison of an item in the array with x .
- Input size: n , the number of items in the array.
- Because $n \geq 1$, there must be at least one pass through the loop, **If $x = S[1]$, there will be one pass through the loop regardless of the size of n .**
- Therefore,

$$B(n) = 1.$$

Algorithm 1.1

Sequential Search

Problem: Is the key x in the array S of n keys?

Inputs (parameters): positive integer n , array of keys S indexed from 1 to n , and a key x .

Outputs: *location*, the location of x in S (0 if x is not in S).

```
void seqsearch (int n,
                const keytype S[ ],
                keytype x,
                int& location)
{
    location = 1;
    while (location <= n && S[location] != x)
        location++;
    if (location > n)
        location = 0;
}
```

Time Complexity

- An average-case analysis is valuable because it **tells us how much time the algorithm would take** when used **many times on many different inputs**.
- This would be useful, for example, in the case of a sorting algorithm that was used repeatedly to sort all possible inputs.
- Often, a relatively slow sort can occasionally be tolerated if, on the average, the sorting time is good. Quicksort(Covered in later lessons) , that does exactly this.
- **Quicksort** is one of the most popular sorting algorithms.

Time Complexity

- As noted previously, an average-case analysis would not suffice in a system that monitored a nuclear power plant.
 - Ex: Life Critical Systems or Mission Critical Systems? State examples
- In this case, a worst-case analysis would be more useful because it would give us an upper bound on the time taken by the algorithm.
- For both the applications just discussed, a best-case analysis would be of little value.

Further examples

Best Case : Best case performance used in computer science to describe an algorithm's behavior **under optimal conditions**.

An example of best case performance would be trying to sort a list that is **already** sorted using some sorting algorithm. e.g. [1,2,3] --> [1,2,3]

Average Case : Average case performance measured using the **average optimal conditions** to solve the problem.

For example a list that is **neither best case nor, worst case order** that you want to be sorted in a certain order. e.g [2,1,5,3] --> [1,2,3,5] OR [2,1,5,3] --> [5,3,2,1]

Worst Case : Worst case performance used to analyze the algorithm's behavior **under worst case** input and least possible to solve the problem.

- It determines when the algorithm will perform worst for the given inputs.

An example of the worst case performance would be a list of names **already sorted in ascending order** that you want to **sort in descending order**. e.g. [1,2,3,5] --> [5,3,2,1]

https://everythingcomputerscience.com/algorithms/Algorithm_Analysis.html

Analysis of an Algorithm

- “analysis of an algorithm” means an **efficiency analysis** in terms of **either time or memory**.
- Algorithms with time complexities such as **n and $100n$** are called **linear-time** algorithms because their time complexities are **linear** in the **input size n** ,
 - whereas algorithms with time complexities such as **n^2 and $0.01n^2$** are called **quadratic time** algorithms because their time complexities are **quadratic** in the **input size n** .

Analysis of an Algorithm ctd..

- Any linear-time algorithm is eventually more efficient than any quadratic-time algorithm.
- In the theoretical analysis of an algorithm, we are interested in eventual behavior.
- Next we will show how algorithms can be grouped according to their eventual behavior.
- In this way we can readily determine whether one algorithm's eventual behavior is better than another's.

An Intuitive Introduction to Order?

- Functions such as $5n^2$ and $5n^2 + 100$ are called pure quadratic functions because they contain no linear term, whereas a function such as $0.1n^2 + n + 100$ is called a complete quadratic function because it contains a linear term. Table shows that eventually the quadratic term dominates this function.

n	$0.1n^2$	$0.1n^2 + n + 100$
10	10	120
20	40	160
50	250	400
100	1,000	1,200
1,000	100,000	101,100



- That is, the values of the other terms eventually become insignificant compared with the value of the quadratic term.

An Intuitive Introduction to Order

- Therefore, although the function is NOT a pure quadratic function, we can classify it with the pure quadratic functions.
- This means that if some algorithm has this time complexity, we can call the algorithm a quadratic-time algorithm.
- Intuitively, it seems that we should always be able to throw away low-order terms when classifying complexity functions.
- For example, it seems that we should be able to classify $0.1n^3 + 10n^2 + 5n + 25$ with pure cubic functions.

Representative Order Functions

$O(\lg n)$

$O(n)$: linear

$O(n \lg n)$: log-Linear

$O(n^2)$: quadratic

$O(n^3)$: cubic

$O(2^n)$: exponential

$O(n!)$: combinatorial

O	Complexity		
$O(1)$	constant	fast	
$O(\log n)$	logarithmic		
$O(n)$	linear time		
$O(n * \log n)$	log linear		
$O(n^2)$	quadratic		
$O(n^3)$	cubic		
$O(2^n)$	exponential		
$O(n!)$	factorial	slow	

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

$O(1)$

Constant Time: Big O of 1

e.g : single statement

$x = x + 2 * y$

$\Rightarrow O(1)$

e.g : sequence of statements

$x = x + 2 * y;$

If $a < 10$ then

$x = x + 2 * y;$

print x;

$\Rightarrow O(1) + O(1) + O(1) = 3 * O(1) = O(1)$

**** Drop Constants**

- **Running time** is independent of input size N

$O(n)$

Linear Time: Big of n

e.g Loop

for $i = 1$ to n

print i;

Print statement $O(1) \Rightarrow$ Loop $n * O(1) = O(N)$

e.g Multiple statements

$x = x + 2 * y;$

for $i = 1$ to n

print i;

$\Rightarrow O(1) + O(N) = O(N)$

**** Drop Low order terms**

Base of Log Matter in Time Complexity?

- In Big-O complexity analysis, it **does not matter what the logarithm base is**. (they are asymptotically the same,
 - i.e. they **differ by only a constant factor**/implies that it is only a **constant factor difference**):
$$O(\log_2 N) = O(\log_{10} N) = O(\log_e N)$$
- Mostly in mathematics it implicitly mean to base ***e***.
- In Computing it tend to favors base 2, but it **does not** actually **matter**.

Common runtimes examples

$O(1)$ **Constant Time**: The algorithm's running time **does not depend on the size of the input**; it performs a fixed number of operations.

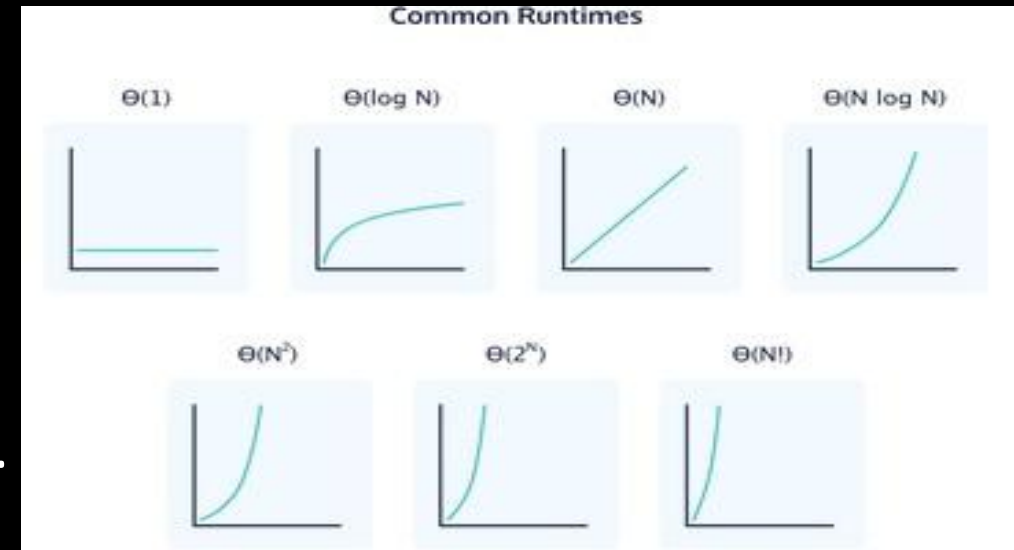
$O(\log n)$ **Logarithmic Time**: The algorithm's running time grows logarithmically with the size of the input.

$O(n)$ **Linear Time**: The algorithm's running time scales linearly with the size of the input.

$O(n \log n)$ **Linearithmic Time**: The algorithm's running time grows in proportion to n times the logarithm of n .

$O(n^2)$ **Quadratic Time**: The algorithm's running time is directly proportional to the square of the input size.

$O(2^n)$ **Exponential Time**: The algorithm's running time doubles with each increase in the input size.



Example

- The quadratic term eventually determines

n	$0.1n^2$	$0.1n^2 + n + 100$
10	10	120
20	40	160
50	250	400
100	1,000	1,200
1,000	100,000	101,100

Growth rate of time complexity

- The growth rate describes the **relationship between input size and the increase in computational cost** (time OR space)
- Growth rate of time complexity is a measure of how fast the **running time** of an algorithm **increases as the input size n increases**.
- It **focuses on the trend of growth** rather than the exact execution time.
- e.g.
 - Constant $O(1)$,
 - Slow growth $O(\log n)$,
 - Linear growth $O(n)$,
 - slightly faster than linear ($n \log n$),
 - Quadratic growth $O(n^2)$,
 - Exponential growth $O(2^n)$

Orders of Growth



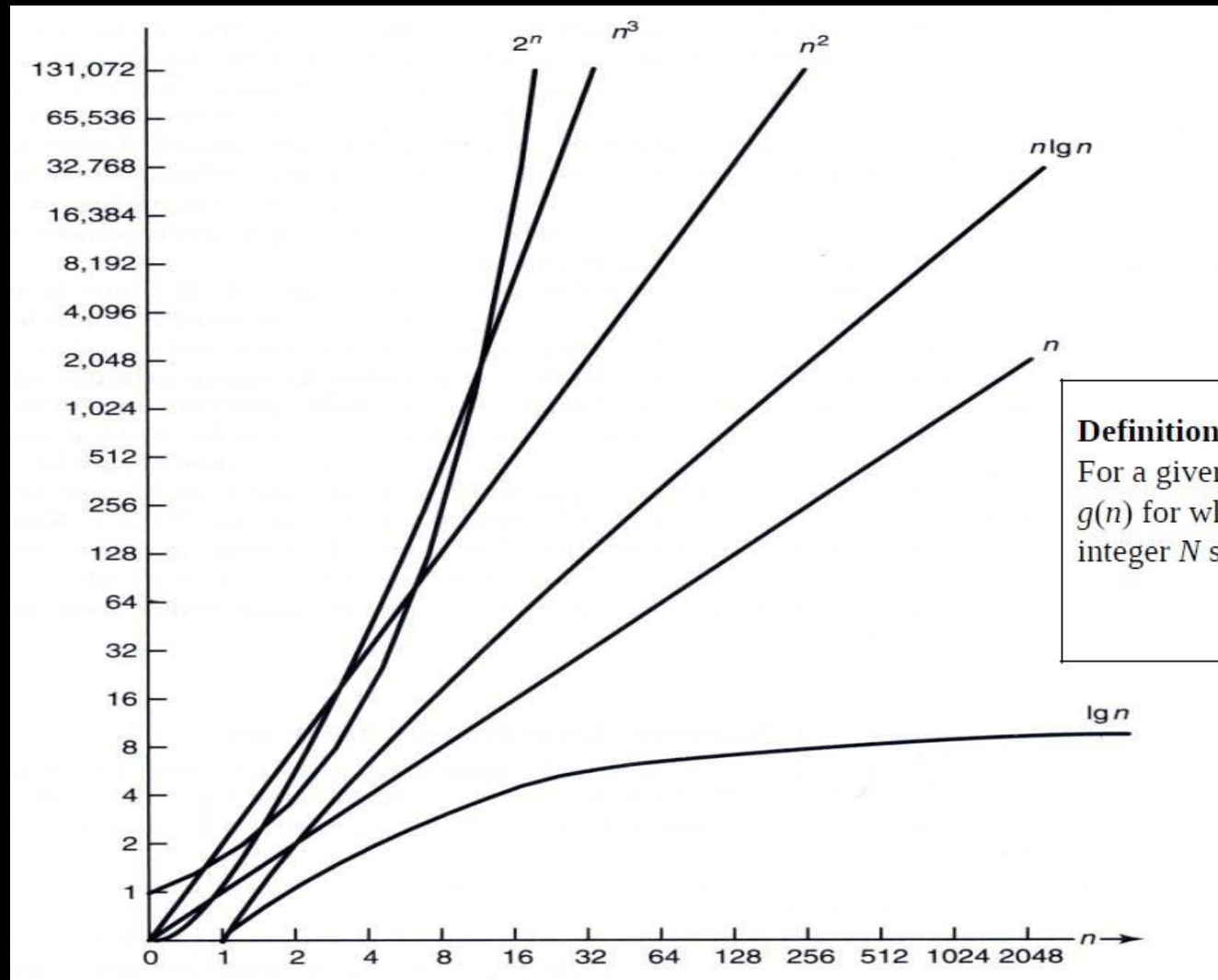
$O(1)$	Constant
$O(\log n)$	Logarithmic
$O(n)$	Linear
$O(n \log n)$	Polynomial
$O(n^2)$	Quadratic
$O(n^3)$	Cubic
$O(2^n)$	Exponential
$O(n!)$	Factorial

Big O

denoted as $O(n)$, is a measure of the growth of the running time of an algorithm proportional to the size of the input.

- The “O” in Big O stands for “order” while the value within parentheses indicates the growth rate of the algorithm.
- In the case of $O(N)$, we refer to it as complexity.
- This implies that the execution time of the algorithm increases proportionally with respect, to the size of the input.
- If we double our input size we can expect twice as much execution time.
- If dropping the non-dominant terms, then how about $n * \log n$?
 - While it is true that we drop the non-dominant terms in Big O, that is generally when we are adding two different complexities, such as $n^2 + n$.
 - Here, we are using multiplication. We can not simplify $n * \log n$ any further, so we keep both terms.

Growth Rates of Some Complexity Functions



$$O(1) < O(\lg n) < O(n) < O(n \lg n) < O(n^2) < O(2^n) < O(n!)$$

Definition

For a given complexity function $f(n)$, $O(f(n))$ is the set of complexity functions $g(n)$ for which there exists some positive real constant c and some nonnegative integer N such that for all $n \geq N$,

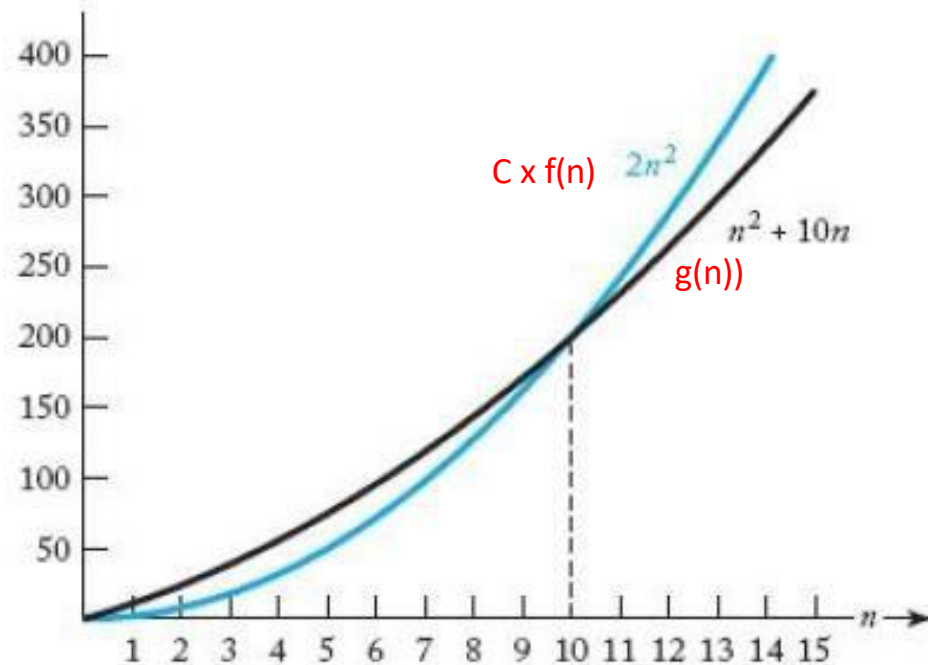
$$g(n) \leq c \times f(n).$$

If $g(n) \in O(f(n))$, we say that $g(n)$ is **big O** of $f(n)$.

Although $g(n)$ starts out above $cf(n)$ in that figure, eventually it falls beneath $cf(n)$ and stays there. Figure 1.5 shows a concrete example. Although $n^2 + 10n$ is initially above $2n^2$ in that figure, for $n \geq 10$

$$g(n) \leq c \times f(n).$$

Figure 1.5 The function $n^2 + 10n$ eventually stays beneath the function $2n^2$.



Example: Execution Times for Algorithms with the Given Time Complexities

n	$f(n) = \lg n$	$f(n) = n$	$f(n) = n \lg n$	$f(n) = n^2$	$f(n) = n^3$	$f(n) = 2^n$
10	$0.003 \mu s^*$	$0.01 \mu s$	$0.033 \mu s$	$0.10 \mu s$	$1.0 \mu s$	$1 \mu s$
20	$0.004 \mu s$	$0.02 \mu s$	$0.086 \mu s$	$0.40 \mu s$	$8.0 \mu s$	$1 \text{ ms}^\dagger$
30	$0.005 \mu s$	$0.03 \mu s$	$0.147 \mu s$	$0.90 \mu s$	$27.0 \mu s$	1 s
40	$0.005 \mu s$	$0.04 \mu s$	$0.213 \mu s$	$1.60 \mu s$	$64.0 \mu s$	18.3 min
50	$0.006 \mu s$	$0.05 \mu s$	$0.282 \mu s$	$2.50 \mu s$	$125.0 \mu s$	13 days
10^2	$0.007 \mu s$	$0.10 \mu s$	$0.664 \mu s$	$10.00 \mu s$	1.0 ms	$4 \times 10^{13} \text{ years}$
10^3	$0.010 \mu s$	$1.00 \mu s$	$9.966 \mu s$	1.00 ms	1.0 s	
10^4	$0.013 \mu s$	$10.00 \mu s$	$130.000 \mu s$	100.00 ms	16.7 min	
10^5	$0.017 \mu s$	0.10 ms	1.670 ms	10.00 s	11.6 days	
10^6	$0.020 \mu s$	1.00 ms	19.930 ms	16.70 min	31.7 years	
10^7	$0.023 \mu s$	0.01 s	2.660 s	1.16 days	$31,709 \text{ years}$	
10^8	$0.027 \mu s$	0.10 s	2.660 s	115.70 days	$3.17 \times 10^7 \text{ years}$	
10^9	$0.030 \mu s$	1.00 s	29.900 s	31.70 years		

Orders of Growth

Good

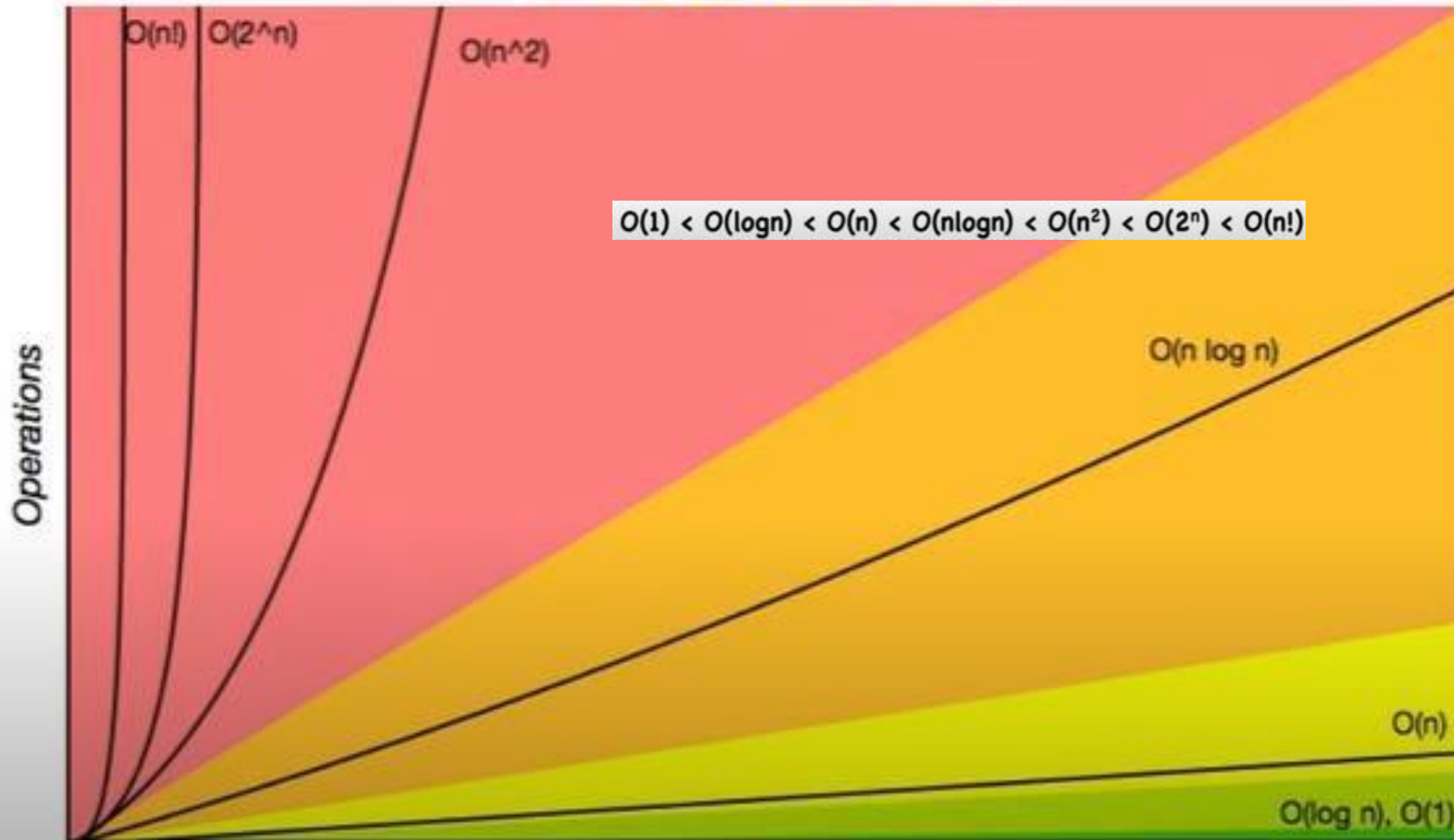


Bad

$O(1)$	Constant
$O(\log n)$	Logarithmic
$O(n)$	Linear
$O(n \log n)$	Polynomial
$O(n^2)$	Quadratic
$O(n^3)$	Cubic
$O(2^n)$	Exponential
$O(n!)$	Factorial

Big-O Complexity Chart

Horrible Bad Fair Good Excellent



Orders of Growth

Good



Bad

$O(1)$	Constant
$O(\log n)$	Logarithmic
$O(n)$	Linear
$O(n \log n)$	Polynomial
$O(n^2)$	Quadratic
$O(n^3)$	Cubic
$O(2^n)$	Exponential
$O(n!)$	Factorial

Big-O notation in 5 minutes

https://www.youtube.com/watch?v=_vX2sjlpXU

Simplification Rules

Simplification Rules

1. Ignore constants: e.g $5n \Rightarrow O(n)$
2. Ignore lower-order terms: e.g $n^2+n+10 \Rightarrow O(n^2)$
3. Keep only the fastest-growing term.

Rules for Analysis

1. Sequential Statements

Add complexities and keep the dominant term.

for i = 1 to n

...

for i = 1 to n

for j = 1 to n

....

$O(n) + O(n^2) \Rightarrow O(n^2)$

2. Nested Loops

- Multiply complexities.

for i = 1 to n

for j = 1 to n

....

Time Complexity: $O(n^2)$

3. Consecutive Loops

- Add complexities.

for i = 1 to n

...

for j = 1 to n

...

Time Complexity: $O(n+n) \Rightarrow O(n)$

Big O?

Simplified Analysis of an Algorithms Efficiency

what is it?

simplified analysis of an algorithm's efficiency

1. complexity in terms of input size, N
2. machine-independent
3. basic computer steps
4. time & space

Type of measurement?

worst-case
best-case
average-case

Big O?

- General Rules

1. ignore constants

$$5n \rightarrow O(n)$$

2. certain terms "dominate" others

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$$

i.e., ignore low-order terms

in practice

1. constants matter
2. be cognizant of best-case and average-case

constant time

$O(1)$ "big oh of one"

```
x = 5 + (15 * 20);
```

independent of input size, N

constant time

```
x = 5 + (15 * 20);
```

```
y = 15 - 2;
```

```
print x + y;
```

total time = $O(1) + O(1) + O(1) = O(1)$
 $3 * O(1)$

linear time

$N * O(1) = O(N)$

```
for x in range (0, n):  
    print x; //  $O(1)$ 
```

```
y = 5 + (15 * 20);           O(1)
for x in range (0, n):      } O(N)
    print x;
```

total time = $O(1) + O(N) = O(N)$

$O(N^2)$

```
x = 5 + (15 * 20);           O(1)
for x in range (0, n):      } O(N)
    print x;
for x in range (0, n):      } O(N^2)
    for y in range (0, n):
        print x * y;
```

$O(N^2)$

```
if x > 0:
    // O(1)
else if x < 0:
    // O(logn)
else:
    // O(n^2)
```

$O(N^2)$

```
for x in range (0, n):
    for y in range (0, n):
        print x * y; // O(1)
```

Examples

- The algorithm complexity **ignores the constant value** in algorithm analysis and **takes only the highest order**.
- Suppose we had an algorithm that takes, **$5n^3+n+4$** time to calculate all the steps, then the algorithm analysis **ignores** all the **lower order polynomials and constants** and takes only **$O(n^3)$** .

Examples ctd..

Simple Statement

This statement takes $O(1)$ time.

```
int y = n + 25;
```

If Statement

The **worst case $O(n)$** if the if statement is **in a loop that runs n times**, best case $O(1)$

```
if( n > 100)
{
    ...
} else {
    ..
}
```

Constant Time Complexity $O(1)$:

The time complexity of a function (or set of statements) is considered as $O(1)$ **if it doesn't contain a loop, recursion, and call to any other non-constant time function.**

i.e. set of **non-recursive and non-loop** statements

Further Examples ctd..

For While Loops

The for loop takes **n time** to complete and so it is **$O(n)$** .

```
for(int i=0; i<n ;i ++)  
{  
  
..  
}
```

Examples ctd..

- The **while loop takes n time** as well to complete and so it is **$O(n)$** .

```
int i=0;
while( i<n)
{
    ..
    i++;
}
```

- If the for **loop takes n time** and i **increases or decreases by a constant**, the cost is **$O(n)$**

```
for(int i = 0; i < n; i+=5)
    sum++;
```

```
for(int i = n; i > 0; i-=5)
    sum++;
```

denoted as $O(n)$, is a measure of the growth of the running time of an algorithm proportional to the size of the input.
In an $O(n)$ algorithm, the running time increases linearly with the size of the input.

Examples ctd..

- If the for loop takes **n time** and **i increases or decreases by a multiple**, the cost is **$O(\log(n))$**

```
for(int i = 1; i <= n; i*=2)  
    sum++;
```

```
for(int i = n; i > 0; i/=2)  
    sum++;
```

The time Complexity of a loop is considered as $O(\log n)$ if the **loop variables are divided/multiplied by a constant amount**. And also for recursive calls in the recursive function, the Time Complexity is considered as $O(\log n)$.

Examples ctd..

Nested loops

If the nested loops contain **sizes n and m , the cost is $O(nm)$**

```
for(int i=0; i<n; i++)  
{  
    for(int i=0; i<m; i++){  
        ..  
    }  
}
```

Ex: What is the time complexity if $C = AB$ for an $n \times m$ matrix A and an $m \times p$ matrix B , (then C is an $n \times p$ matrix with entries)

Input: matrices A and B

Let C be a new matrix of the appropriate size

For i from 1 to n :

- For j from 1 to p :
 - Let $\text{sum} = 0$
 - For k from 1 to m :
 - Set $\text{sum} \leftarrow \text{sum} + A_{ik} \times B_{kj}$
 - Set $C_{ij} \leftarrow \text{sum}$

Return C

Refer: <https://www.geeksforgeeks.org/how-to-analyse-loops-for-complexity-analysis-of-algorithms/>

Examples ctd..

- If the **first loop runs n times** and the **inner loop runs $\log(n)$ times** or (vice versa), the cost is **$O(n * \log(n))$**

```
for(int i=0; i<n ; i++)  
{  
    for(int j=1; j<=n; j*=2){  
        ..  
        ..  
    }  
}
```

Examples ctd..

- If the **first loop runs n^2** times and the **inner loop runs n** times or (vice versa), the cost is $O(n^3)$

```
for(int j=0 ;j<n*n ;j++)  
{  
    for(int i=0 ;i<n ;i++){  
        ..  
        ..  
    }  
}
```

Examples ctd..

- If the first loop runs n times and the inner second loop runs n^2 times and the third loop runs n^2 , then $O(n^5)$

```
for(int i = 0; i < n; i++)  
  for( int j = 0; j < n * n; j++)  
    for(int k = 0; k < j; k++)  
      sum++;
```

Ex: What is $O(?)$

```
1: for  $i = n$  to  $3n^2 + 4n$  do  
2:   for  $j = 1$  to  $7n + 3$  do  
3:      $x = x + i - j$   
4:   end for  
5: end for
```

$3n^2 + 4n$
 $-n + 1$
times

```
1: for  $i = n$  to  $3n^2 + 4n$  do  
2:   for  $j = 1$  to  $7n + 3$  do  
3:      $x = x + i - j$   
4:   end for  
5: end for
```

c } $7n + 3$ times

Runtime: $(3n^2 + 3n + 1)(7n + 3)c$
 $\approx cn^3$

References

- [Refer Asymptotic Notations 101: Big O, Big Omega, & Theta \(Asymptotic Analysis Bootcamp\)](https://www.youtube.com/watch?v=0oDAIMwTrLo&t=50s)
 - <https://www.youtube.com/watch?v=0oDAIMwTrLo&t=50s>

